Chapter 28

Quiz Questions & Detailed Answers

This chapter provides a comprehensive set of questions designed to test and reinforce your understanding of the material covered throughout this guide. Each question targets a key concept, algorithm, or system design decision—the kind of knowledge that distinguishes surface-level familiarity from genuine expertise. Use these questions for self-assessment: attempt your own answer before reading the detailed solution. The questions progress from foundational concepts (LLM architecture, reinforcement learning basics) through core algorithms (PPO, DPO, GRPO) to advanced system design and agentic AI topics.

28.1 Foundations Questions

Q0a: What is the role of the attention mechanism in a decoder-only Transformer? Why is it causal?

Answer: The attention mechanism allows each token to attend to (i.e., compute a weighted combination of) representations from other tokens. In a decoder-only Transformer, attention is **causal** (also called *autoregressive*): token t can only attend to tokens $1, \ldots, t$, never to future tokens $t+1, \ldots, T$.

Why causal? Because the model generates text left-to-right. At inference time, future tokens literally do not exist yet. The causal mask during training simulates this constraint so that the model learns to predict each token using only its left context. Mathematically, the attention matrix is masked:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

where $M_{ij} = -\infty$ for $j > i$ (future positions), forcing those attention weights to zero.

Practical implication: This enables the KV-cache optimization at inference—since past tokens' keys and values never change, they can be cached and reused, reducing generation from $O(T^2)$ to $O(T)$ per new token.

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q0b: Explain Flash Attention. What problem does it solve and how?

Answer: Standard attention computes the full $T \times T$ attention matrix, which requires $O(T^2)$ memory and is **memory-bandwidth bound**—the GPU spends most of its time moving data between HBM (slow, large) and SRAM (fast, small), not doing actual computation.

Flash Attention's insight: Never materialize the full attention matrix in HBM. Instead, tile the computation into blocks that fit in SRAM, compute attention *block by block* using an online softmax algorithm, and write only the final output to HBM.

Key techniques:

1. **Tiling**: Split Q, K, V into blocks of size $B_r \times B_c$ that fit in SRAM
2. **Online softmax**: Track running max and sum to compute softmax incrementally without

the full row

3. **Recomputation**: In the backward pass, recompute attention from Q, K, V (cheap) rather than storing the $T \times T$ matrix (expensive)

Result: $O(T)$ HBM memory (instead of $O(T^2)$), 2–4× wall-clock speedup, exact same numerical output (not an approximation).
Review: *Chapters 1–2 (LLM Architecture; Systems Foundations).*

Q0c: What is the difference between SFT, RLHF, and DPO at a high level? When do you use each?

Answer:

- **SFT (Supervised Fine-Tuning)**: Train the model to imitate high-quality demonstrations. Loss: next-token prediction on curated data. Teaches *format* and *style*.
- **RLHF**: Train a reward model from human preferences, then optimize the policy against it using RL (PPO). The model explores beyond the demonstration data. Teaches *what humans prefer*.
- **DPO**: Skip the reward model. Directly optimize the policy on preference pairs (y_w, y_l) using a contrastive loss. Same goal as RLHF but simpler pipeline.

Typical pipeline: SFT first (gives the model a good starting point), then RLHF or DPO (refines preferences). SFT alone tends to produce verbose, hedge-heavy responses. RLHF/DPO makes outputs more direct and aligned with human intent.
When to use each: SFT when you have gold-standard outputs. DPO when you have preference pairs but limited compute. RLHF (PPO) when you need maximum quality and can afford the infrastructure.
Review: *Chapters 5, 6, and 10 (PPO; DPO; SFT Best Practices).*

Q0d: What is a reward model? How is it trained and what can go wrong?

Answer: A reward model (RM) is a neural network that takes a (prompt, response) pair and outputs a scalar score indicating quality. It is trained on human preference data: given pairs (y_w, y_l) where y_w is preferred, the RM learns to assign $R(y_w) > R(y_l)$.
Training: Bradley-Terry loss: $\mathcal{L} = -\log \sigma(R(y_w) - R(y_l))$. Architecture: typically the same transformer as the policy, with the LM head replaced by a scalar projection.
What can go wrong:

1. **Reward hacking**: The policy finds outputs that score high on the RM but are actually low quality (e.g., excessively long, repetitive, or containing specific phrases the RM was biased toward)
2. **Distribution shift**: The RM was trained on outputs from an earlier policy. As training progresses, the current policy generates out-of-distribution outputs the RM cannot score accurately
3. **Label noise**: Human annotators disagree, are tired, or apply inconsistent criteria. This noise propagates into RM predictions
4. **Overconfidence**: The RM assigns extreme scores to outputs it has never seen, providing misleading gradient signal

Review: *Chapter 9 (Reward Model Training).*

Q0e: Explain the exploration-exploitation trade-off in RL. How does it manifest in LLM training?

Answer: In RL, an agent must balance:

- **Exploitation**: choosing actions known to yield high reward (greedy behavior)
- **Exploration**: trying new actions that might yield even higher reward (but might also fail)

In LLM training: The policy is the language model. "Actions" are token choices. "Exploitation" means generating responses similar to what already scored well. "Exploration" means trying novel phrasings, structures, or reasoning paths.
How it manifests:

- **Temperature during generation**: Higher temperature = more exploration. GRPO uses temperature 1.0 to get diverse samples within each group.
- **KL penalty**: Acts as an anti-exploration brake—prevents the policy from straying too far from the reference model. Without it, the policy might collapse to a single high-reward template (mode collapse).
- **Group sampling in GRPO**: Generating G responses per prompt explicitly explores the output space, then reinforces above-average responses.

The tension: Too little exploration → the model gets stuck in local optima (always giving the same safe answer). Too much → training is unstable, quality fluctuates wildly.
Review: *Chapters 3 and 7 (Introduction to RL; GRPO).*

28.2 Core Algorithm Questions

Q1: Explain PPO's clipped objective. Why does it work better than vanilla PG?

Answer: Vanilla policy gradient: $\nabla J = \mathbb{E}[\nabla \log \pi(a|s) \cdot \hat{A}]$. Problem: one lucky/unlucky sample can produce a huge gradient → policy jumps to a bad region → generates garbage → next gradient makes it worse → unrecoverable "death spiral."
PPO's solution: Clip the probability ratio $r = \pi_{\text{new}}/\pi_{\text{old}}$ to $[0.8, 1.2]$.
Mechanics: For good actions ($\hat{A} > 0$): objective is $\min(r\hat{A}, 1.2\hat{A})$. Once r exceeds 1.2, no further benefit — stops the policy from over-committing to one example. For bad actions ($\hat{A} < 0$): objective is $\min(r\hat{A}, 0.8\hat{A})$. Once r drops below 0.8, penalty stops growing — prevents catastrophic forgetting.
Key insight: It's a first-order approximation of TRPO's KL constraint, but without expensive second-order optimization. Each update changes the policy by at most $\pm 20\%$.
For LLMs specifically: The token-level ratio $r_t = \pi_\theta(y_t|y_{<t})/\pi_{\text{old}}(y_t|y_{<t})$ prevents any single token's probability from changing too drastically, preserving coherent generation.
Review: *Chapter 5 (PPO).*

Q2: Derive DPO from first principles. What assumptions does it make?

Answer: Start with RLHF objective: $\max_\pi \mathbb{E}[r(x,y)] - \beta D_{\text{KL}}[\pi \| \pi_{\text{ref}}]$.
Step 1: Write the KKT conditions. Optimal policy has closed form: $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(r(x,y)/\beta)$.
Step 2: Invert to express reward: $r(x,y) = \beta \log(\pi^*/\pi_{\text{ref}}) + \beta \log Z(x)$.
Step 3: Substitute into Bradley-Terry model $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$. The partition function $Z(x)$ cancels (same prompt).
Step 4: Replace π^* with π_θ (parameterized policy we're training): $\mathcal{L} = -\mathbb{E}[\log \sigma(\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)})]$.

Assumptions:

1. Bradley-Terry preference model (pairwise, no ties, transitive)
2. Optimal policy is achievable by π_θ (sufficient capacity)
3. Preferences are generated from the same distribution as training data (no distribution shift)
4. Reference model is fixed and reasonable

When assumptions break: Real preferences aren't transitive, data shifts during training, labels are noisy → that's why Online DPO and IPO exist.
Review: *Chapter 6 (DPO).*

Q3: GRPO vs PPO — when would you choose each? What's the trade-off?

Answer:
GRPO advantages:

- No value function needed: saves one model's worth of memory and complexity
- Simpler: fewer hyperparameters, more intuitive (above-mean = good, below-mean = bad)
- Better for verifiable rewards: math/code where $r \in \{0, 1\}$ gives crisp signal
- DeepSeek-R1 proved it can teach emergent reasoning with just binary rewards

PPO advantages:

- Per-token credit assignment: value function assigns reward to each token, not just sequence-level
- More sample efficient: GAE uses value predictions to estimate advantage without generating G samples
- Better for nuanced rewards: when reward is continuous and varies significantly across tokens
- More mature: battle-tested at OpenAI, Anthropic, etc.

Rule of thumb: If rewards are verifiable (right/wrong) → GRPO. If rewards are nuanced (RM scores) and you need max quality → PPO. If compute is limited → GRPO (no critic training).
Compute comparison: GRPO generates G responses per prompt (8× more generation), but skips value function training. Net: similar total compute but distributed differently (more gen, less training).
Review: *Chapters 5 and 7 (PPO; GRPO).*

Q4: How does GAE work? Walk through a concrete example for LLMs.

Answer: GAE = weighted sum of n-step TD errors: $\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}$.
Concrete example: Response has 5 tokens. Reward only at end ($r_5 = 0.8$). Value predictions: $V_1 = 0.5, V_2 = 0.55, V_3 = 0.6, V_4 = 0.65, V_5 = 0.7$.
TD errors ($\gamma = 1$): $\delta_1 = 0 + V_2 - V_1 = 0.05$, $\delta_2 = 0 + V_3 - V_2 = 0.05$, ..., $\delta_5 = 0.8 + 0 - 0.7 = 0.1$.
With $\lambda = 0.95$: $\hat{A}_5 = 0.1$ (just the final TD error), $\hat{A}_4 = 0.05 + 0.95 \times 0.1 = 0.145$, $\hat{A}_3 = 0.05 + 0.95 \times 0.145 = 0.188$, etc.
Interpretation: Token 3 gets advantage 0.188 because it contributed to a sequence that got higher reward than expected. Earlier tokens get credit through the exponential decay.
For LLMs: $\gamma = 1.0$ (all tokens matter, finite horizon). The advantage at token t answers: "given what followed this token, was this token choice better or worse than expected?"
Review: *Chapter 5 (PPO).*

Q5: How do you prevent reward hacking? Give a layered defense strategy.

Answer: **Detection signals**: RM score rising but win-rate flat/declining. Response length growing monotonically. KL divergence > 15. Diversity (unique n-grams) dropping. Reading high-reward outputs reveals exploits.
Layered defense (in priority order):

1. **KL penalty** (primary): Adaptive controller targets KL ≈ 6. If KL rises, β increases automatically. Prevents drifting too far from reference.
2. **Reward model ensemble** (3–5 models): Use min or mean of scores. Individual models have different blind spots — exploits that fool one rarely fool all.
3. **Length penalty**: $r' = r - c \cdot \max(0, \text{length} - L_{\text{target}})$. Prevents "just generate longer = higher score" exploit.
4. **Periodic RM refresh**: Every 2000 steps, generate data from current policy, relabel, add to RM training set. Closes the exploit as model finds it.
5. **Win-rate based stopping**: Track win-rate against SFT baseline. If RM score rises but win-rate stalls for 200+ steps, stop training. The model is exploiting, not improving.

Post-detection recovery: Roll back to last "clean" checkpoint. Increase β by $2\times$. Add the discovered exploit to the RM's negative examples.
Review: *Chapters 9 and 11 (Reward Model Training; System Architecture).*

28.3 System Design Questions

Q6: Design an RLHF system for training a 70B model. Walk through every component.

Answer: Three-cluster decoupled architecture on 72 A100-80GB GPUs:
Cluster 1 — Generation (32 GPUs):

- 8 vLLM instances, TP=4 each. PagedAttention + speculative decoding (1B draft model).
- Continuous batching, max 256 sequences in flight. INT8 weights for bandwidth.
- Output: (prompt, response, per-token log-probs). Throughput: $\sim$500 responses/minute.
- Stateless: just loads latest weights from shared store. Trivial recovery.

Cluster 2 — Scoring (8 GPUs):

- Reward model (70B, INT8 = 70GB) on 4 GPUs (TP=4).
- Reference model (70B, INT8) on 4 GPUs (TP=4). Computes per-token log-probs for KL.
- Output: reward scores + KL per token. Lightweight, batch inference.

Cluster 3 — Training (32 GPUs):

- Policy model with FSDP (ZeRO-3). Flash Attention 2. Gradient checkpointing.
- Consumes scored experiences from buffer. PPO update: 4 epochs on mini-batch of 16.
- Pushes updated weights to shared store every 50 steps (async, background transfer).
- Async checkpoint every 100 steps (non-blocking write to NVMe + S3 backup).

Connection fabric:

- Gen → Score: Ray/Redis queue (∼10 MB per batch: token IDs + log-probs)
- Score → Train: Experience buffer (circular, holds last 500 steps)
- Train → Gen: Weight store on shared parallel FS (Lustre/GPFS). 140GB push every 50 steps, async.

Overlap: While training processes step N, generation is already producing data for step $N+1$. This hides generation latency, achieving 1.3–1.5× throughput vs monolithic.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q7: How do you handle the generation bottleneck? Quantify the solutions.

Answer: Generation = 60–70% of total RLHF wall-clock time. Root cause: autoregressive decoding is memory-bandwidth bound (arithmetic intensity ≈ 1 FLOP/byte vs roofline of 156 FLOP/byte on A100).
Solutions ranked by impact:
1. Decouple gen from training (1.3–1.5× end-to-end): Run generation on separate hardware, overlap with training. The single biggest architectural win.
2. vLLM with PagedAttention (2–4×): Eliminates 60–80% KV cache memory waste from internal fragmentation. Enables 3–4× larger batches = better bandwidth utilization.
3. Continuous batching (1.5–2×): Don't wait for longest sequence to finish. Start new sequences immediately in freed slots. Keeps GPUs busy.
4. Speculative decoding (2–3×): 1B draft model proposes 5 tokens. 70B model verifies all 5 in one forward pass (parallel!). Accept 3–4 on average → 3–4 tokens per forward pass instead of 1.
5. INT8/FP8 for generation weights (2×): Halves the 140GB weight read per token. Quality loss is minimal because (a) we're sampling with temperature anyway, and (b) only generation uses INT8, training stays BF16.
6. CUDA graphs + kernel fusion (1.1–1.3×): Eliminate Python/CUDA launch overhead. Fuse layernorm+attention+MLP into fewer kernels.
Combined: $1.5 \times 3 \times 1.5 \times 2.5 \times 2 \times 1.2 = 40\times$ over naive. In practice, diminishing returns limit total to ∼10–20× over naive implementation.
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

Q8: Explain weight synchronization in a decoupled system. How much staleness can you tolerate?

Answer:
The problem: Generation cluster uses policy weights to produce responses. Training cluster updates those weights. They're on different hardware. How to keep them in sync?
Why perfect sync is wasteful: Full sync of 70B BF16 = 140GB. At InfiniBand 400Gb/s (50GB/s): 2.8s per sync. If you sync every step (every 50–90s), you spend 3–5% of time on weight transfer. Acceptable, but unnecessary.
Staleness tolerance analysis:

- Per-step policy change: ∼0.1% (measured by mean param delta)
- 50 steps: ∼5% cumulative drift
- PPO clip range: handles up to 20% probability ratio deviation
- Empirical: 50-step staleness → <2% quality degradation (measured by win-rate)

Production strategy:

1. Every 50 training steps: push full BF16 checkpoint to shared store (2.8s transfer)
2. Generation cluster: non-blocking weight reload between batches

3. Delta compression (optional): Only send changed parameters (INT8 delta ≈ 5GB), apply as offset. 10× less bandwidth.

4. For very large scale (256+ GPUs): streaming sync — continuously send small chunks in background. Average staleness: 5–10 steps.

Important subtlety: Log-probs computed during generation use stale weights. The PPO ratio $\pi_{\text{new}}/\pi_{\text{old}}$ computes π_{old} using these stale log-probs. This is fine because PPO is designed for off-policy corrections.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q9: How would you make this fault-tolerant at 512 GPUs?

Answer: At 512 GPUs, MTBF is 4–8 hours. A 5-day training run will see 15–30 failures.
Architecture-level resilience:

- Generation cluster = stateless. Failed instance restarts in <60s (just load weights, no state).
- Training cluster = stateful. Needs checkpoint-based recovery.
- Scoring cluster = stateless. Same as generation.

Checkpointing strategy:

- Frequency: Every 50–100 steps (5–10 min of training).
- Method: Async (non-blocking). Background thread writes while next step proceeds. Uses FSDP's distributed save (each rank saves its shard in parallel).
- Contents: Model weights, optimizer states (Adam m/v), LR scheduler, RNG states, KL adaptive coefficient, global step counter, replay buffer pointer.
- Storage: Local NVMe (fast, 30s for 70B) + async copy to S3/shared FS (durable).
- Retention: Keep last 3 checkpoints. Auto-delete older ones.

Detection and recovery flow:

1. NCCL collective timeout (60s) or heartbeat miss (10s) → failure detected.
2. Identify failed node(s) via NVML health check.
3. Option A (fast, <2 min): Torch Elastic shrinks world size, redistributes shards, continue with $N - 1$ nodes. Request replacement in background.
4. Option B (clean, ∼5 min): Bring up replacement node, rebuild process group, load last checkpoint, resume.
5. Experience buffer is persisted — no regeneration needed.

Prevention: Pre-screening stress test (GEMM, memory, NVLink). ECC error monitoring (preemptive migration if errors spike). Hot spare nodes (pre-loaded with environment). Dual-rail InfiniBand for network redundancy.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q10: How do you scale from 7B to 70B to 405B? What changes at each scale?

Answer:
7B (single 8-GPU node, hours):

- Architecture: Monolithic (TRL default). All models on same GPUs.
- Memory: LoRA + INT8 ref/RM fits in 8×80GB easily.
- Parallelism: DP=8 or FSDP within node. No network communication.
- Hyperparams: LR=5×10^{-6}, aggressive β=0.02, 50K–100K steps.
- Time: 4–12 hours per run. Fast iteration.

70B (32–64 GPUs, 2–5 days):

- Architecture: Semi-decoupled. vLLM generation + FSDP training.
- Memory: ZeRO-3 essential. Gradient checkpointing. INT8 ref/RM.
- Parallelism: TP=8 intra-node (gen), FSDP inter-node (training).
- Hyperparams: LR=1.5×10^{-6}, moderate β=0.05, 10K–30K steps.
- Fault tolerance: Async checkpoints, monitoring, but manageable manually.

405B (256–512 GPUs, 1–3 weeks):

- Architecture: Fully decoupled. Separate clusters. Weight store + queue.
- Memory: ZeRO-3 + TP=8 + PP=2 for training. INT4 generation.
- Parallelism: 3D parallelism (TP×PP×DP = 8×2×16 = 256 GPUs training).
- Hyperparams: LR=5×10^{-7}, very conservative β=0.1, 2K–5K steps.
- Fault tolerance: Mandatory. Elastic training, redundant checkpoints, hot spares.
- Key change: Much less RL training needed (model is already very capable from pretraining). But each step is 50× more expensive, so instability is catastrophic.

Paradox: Larger models are actually *easier to RL-train per step* (more stable, smoother loss landscape). But the cost of instability scales with model size — a bad run at 405B wastes $100K+ in compute.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

28.4 Practical and Debugging Questions

Q11: Reward score is increasing but model quality is declining. Diagnose and fix.

Answer: Classic **reward hacking / Goodhart's Law**.
Diagnostic protocol:

1. **Check response length**: Plot mean length over training. Growing monotonically? = Length exploit (RM gives higher scores to longer responses).
2. **Check KL divergence**: >15? = Policy has diverged too far from reference. Lost capabilities.
3. **Check diversity**: Unique trigrams per response. Dropping? = Mode collapse (repeating same high-reward pattern).
4. **Manual inspection**: Read 20 highest-reward responses. What pattern do they share? (e.g., all start with "Great question!", all use bullet points, excessive hedging).

5. **Win-rate**: Evaluate against SFT baseline on held-out prompts. If flat/declining while RM rises = confirmed exploit.

Immediate fixes:

- Roll back to last checkpoint where win-rate was improving
- Increase β by 2–3× (stronger KL penalty)
- Add explicit length penalty: $r' = r - 0.001 \cdot \max(0, \text{len} - 500)$

Structural fixes (prevent recurrence):

- RM ensemble: Train 3–5 RMs on different data splits. Use min or mean. Exploits are model-specific.
- RM refresh: Every 2000 steps, generate from current policy, get human labels, retrain RM.
- Multi-objective reward: Combine helpfulness + harmlessness + conciseness with separate RMs.
- Early stopping on win-rate (not RM score). The metric you optimize should differ from training signal.

Review: *Chapters 9 and 11 (Reward Model Training; System Architecture).*

Q12: How do you decide the prompt distribution for RL training?

Answer: Prompt quality is the most underrated factor in RLHF. Bad prompts = no learning signal.

Composition (my default mix):

- 40% real user traffic (represents actual use cases)
- 30% synthetic (LLM-generated, fills gaps in coverage — rare topics, edge cases)
- 20% curriculum (graduated difficulty — start easy, increase complexity as model improves)
- 10% adversarial (red-team prompts, jailbreak attempts, ambiguous instructions)

Critical: The Goldilocks Filter:

1. For each candidate prompt, generate 4–8 responses with current model.
2. Score with RM. Compute pass rate (fraction above threshold).
3. Keep only prompts with 20–80% pass rate:
   - <20%: Too hard. Model almost always fails → all negative advantages → no useful gradient.
   - >80%: Too easy. Model almost always succeeds → all positive advantages → no contrast.
   - 20–80%: Perfect. Mix of successes and failures → clear signal about what works.
4. Re-filter every 500 training steps (model improves, difficulty distribution shifts).

Topic diversity: Ensure no single topic dominates (<10% per category). Use embedding clustering to verify coverage. Otherwise the model over-optimizes for dominant topics.

Review: *Chapters 7 and 9 (GRPO; Reward Model Training).*

Q13: LoRA vs full fine-tuning for RLHF. When would you use each?

Answer:
LoRA (Low-Rank Adaptation, r=64, α=16):

- Trainable params: ∼0.2% of model (200M for 70B)
- Memory savings: No separate reference model needed (base model = reference)! Saves 140GB.
- Stability: Inherently more stable (low-rank constraint limits how far policy can drift)
- Speed: Faster per-step (fewer params to update), but may need more steps
- Quality ceiling: 90–95% of full FT quality typically

Full fine-tuning:

- All parameters updated. Maximum expressiveness.
- Needs separate reference model copy (140GB for 70B). Or very frequent checkpointing to "anchor."
- Higher risk of catastrophic forgetting. Need lower LR (3× lower than LoRA) and stronger β.
- Better when: large distributional shift needed (new language, very different style), LoRA hits capacity limit.

My decision framework:

1. Start with LoRA (r=64). It's 3× cheaper and more stable.
2. Monitor gradient norms on LoRA matrices. If consistently >1.0 (high relative to parameter count): LoRA is capacity-limited.
3. Switch to full FT only when: win-rate plateaus AND gradient analysis suggests capacity limitation.
4. For full FT: use LR/3, $\beta \times 2$, more frequent checkpointing, and early stopping based on win-rate.

Hybrid: LoRA for alignment/safety (small behavioral shift) + Full FT for capabilities/reasoning (large shift needed).
Review: *Chapters 1 and 10 (LLM Architecture; SFT Best Practices).*

Q14: Process Reward Models (PRM) vs Outcome Reward Models (ORM). Design a PRM system.

Answer:
ORM: Scores the final answer only. "Is the response good overall?" Simple but can't identify *where* reasoning went wrong.
PRM: Scores each intermediate step. "Is step 3 of this derivation correct?" Much more informative but harder to train.
PRM advantages for reasoning:

- Identifies exactly where reasoning fails (step-level credit assignment)
- Enables tree search: expand only branches with high step scores
- Less reward hacking: can't get high score from wrong steps + lucky final answer

- PRM + best-of-N beats ORM + best-of-N by 10–20% on MATH benchmarks

Training a PRM:

1. **Data collection** (Monte Carlo approach):
   - For each problem, generate reasoning trace step by step
   - At each step k, complete the solution M times ($M = 32$) from that point
   - Step score = fraction of completions that reach correct answer
   - Steps where score drops significantly = the "mistake" steps
2. **Labeling**: Step is "correct" if its completion rate $> 50\%$, "incorrect" if $< 20\%$.
3. **Model**: Same architecture as base model + classification head per token position. Train with binary cross-entropy on step labels.
4. **Inference**: Score each step. If any step scores <0.3, flag the trace as flawed.

Using PRM in RLHF: Per-token rewards from PRM feed directly into GAE. Each token gets immediate feedback, not just end-of-sequence. This dramatically improves credit assignment for long reasoning chains.
Review: *Chapters 9 and 13 (Reward Model Training; RL for Large Reasoning Models).*

Q15: How do you evaluate whether RL actually improved the model?

Answer: Multi-faceted evaluation (no single metric captures "quality"):
1. **Win-rate** (most important, most reliable):
   - 500+ diverse prompts. LLM judge (GPT-4 or Claude) picks winner in blind A/B comparison vs SFT baseline.
   - Target: >55% win-rate = meaningful improvement. >65% = strong improvement.
   - Use position-debiasing (swap A/B order, average). Report confidence intervals.
2. **Capability benchmarks** (regression detection):
   - MMLU (knowledge), HumanEval (code), MATH (reasoning), MT-Bench (multi-turn).
   - Any >2% drop = concerning alignment tax. Investigate which categories degraded.
3. **Category-specific evals**:
   - Safety: refusal rate on harmful prompts (should increase)
   - Truthfulness: TruthfulQA score (should increase or stay flat)
   - Helpfulness: task completion rate on instruction-following benchmarks
4. **Distributional metrics**:
   - Response length distribution (shouldn't shift dramatically)
   - Vocabulary diversity (unique tokens per response)
   - Format compliance (if trained for specific format)
5. **Human evaluation** (gold standard, expensive):

- Blind A/B with 3+ skilled annotators per example. Inter-annotator agreement $> 70\%$.
- Only for final model selection, not during training (too slow/expensive).

Red flags: RM score up + win-rate flat = reward hacking, not real improvement. Win-rate up + benchmarks down = alignment tax too high, reduce RL strength.
Review: *Chapter 14 (LLM Evaluation).*

Q16: Describe the reward model training pipeline end-to-end.

Answer:
Phase 1 — Data Generation:

- Collect 50K–100K diverse prompts (real traffic + synthetic)
- Generate 4–8 responses per prompt at varying temperatures (0.3, 0.7, 1.0) and from multiple models (diversity is key — if all responses are similar, preferences are uninformative)
- Total: 200K–800K candidate responses

Phase 2 — Preference Collection:

- Option A (expensive, best quality): Human annotators compare pairs. 3 annotators per pair. Cost: \$2–5 per comparison.
- Option B (cheap, 85–90% agreement with humans): LLM judge (GPT-4/Claude). 10× cheaper. Good for scale.
- Format: (prompt, chosen response, rejected response). Pairs with annotator disagreement (<70% agreement) are discarded.
- Final dataset: 100K–500K pairs.

Phase 3 — Training:

- Architecture: Same as base LLM + scalar head (one regression output per sequence).
- Loss: $\mathcal{L} = -\mathbb{E}[\log \sigma(r(x, y_w) - r(x, y_l))]$ (Bradley-Terry).
- Training: **1 epoch only!** RMs overfit extremely fast. Validation accuracy 68–75% is good (higher often means overfitting to annotation artifacts).
- Tricks: Center rewards around 0 (subtract running mean). Check for length bias (if correlation between length and score > 0.3, add length penalty to training).

Phase 4 — Validation:

- Hold-out preference pairs: accuracy should be 68–75%.
- Agreement with humans on new data: $> 80\%$.
- Length bias check: correlation between response length and RM score should be < 0.2.
- Consistency check: same prompt, paraphrased responses should get similar scores.

Review: *Chapter 9 (Reward Model Training).*

Q17: What happens when KL divergence explodes? Root cause and fix.

Answer:
What KL measures: Average log-ratio between current policy and reference: $D_{\text{KL}} = \mathbb{E}_{y\sim\pi_\theta}[\log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))]$. KL=0 means identical to reference. KL=10 means the policy puts 10 nats more probability on its preferred outputs.
Healthy range: 3–10 during training. Slowly increasing is fine. Sudden spike = problem.
Root causes of KL explosion:

1. **Learning rate too high**: Policy takes giant step, diverges from reference. Fix: reduce LR by 2–5×.
2. **Reward hacking**: Found an exploit that gets high reward far from reference behavior. Fix: increase β, add RM ensemble.
3. **Mode collapse**: Policy concentrates on one response template. KL is high at that template, low everywhere else. Fix: increase entropy bonus, increase temperature.
4. **Bad batch**: One unlucky batch with extreme advantages pushed the policy. Fix: gradient clipping, reduce mini-batch size.
5. **Value function diverged**: Wrong advantage estimates cause wrong updates. Fix: reduce value function LR, or switch to GRPO (no value function).

Recovery protocol:

1. Detect: KL $>$ 15 for 50+ steps, or KL jumps $>$5 in one step.
2. Immediate: Load last clean checkpoint (KL $<$ 10).
3. Adjust: Reduce LR by 50%. Increase β by 2×. Lower cliprange to 0.1.
4. Resume: Monitor closely for first 200 steps.

Review: *Chapters 5 and 7 (PPO; GRPO).*

Q18: Compare monolithic vs decoupled RLHF architectures. When does each make sense?

Answer:
Monolithic (TRL default: single process, all models on same GPUs):

- Pros: Simple code. No distributed systems complexity. Easy debugging.
- Cons: GPUs idle 60% of time (compute idle during gen, bandwidth idle during training). Doesn't scale past ∼16 GPUs efficiently. All models compete for same memory.
- Use when: Model $\leq$ 13B, single node, research/prototyping.

Semi-decoupled (vLLM gen + FSDP training, same cluster):

- Pros: Better utilization (gen and training can partially overlap). Scales to 64 GPUs.
- Cons: Still shares hardware, can't optimize independently. More complex than monolithic.
- Use when: 13B–70B, 2–8 nodes, production experiments.

Fully decoupled (separate clusters connected by queues):

- Pros: Each cluster optimized for its workload. Gen scales independently from training. Gen cluster is stateless (trivial fault tolerance). Scales to hundreds of GPUs.

- Cons: Distributed systems complexity. Weight staleness. Queue management. Network overhead.
- Use when: $\geq$ 70B production training. Need scale, fault tolerance, high utilization.

The key insight: Generation is bandwidth-bound, training is compute-bound. Same hardware can't optimize for both. Decoupling lets gen nodes have: more memory bandwidth, INT8 weights, large batch. Training nodes have: full BF16 precision, Flash Attention, FSDP sharding.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q19: How do you set up curriculum learning for RL training?

Answer: Curriculum = gradually increasing difficulty so the model learns progressively.
Why it matters: If you throw the hardest prompts at a weak model, it gets all-negative rewards $\rightarrow$ no learning signal (everything is equally bad). If you start easy, the model develops basic capabilities, then builds on them.
Implementation:

1. **Difficulty scoring**: Rate each prompt by current model's pass rate (from Goldilocks filtering). Easy = >80% pass, Medium = 30–80%, Hard = <30%.
2. **Schedule**: Steps 0–1000: 70% easy, 20% medium, 10% hard. Steps 1000–5000: 30% easy, 50% medium, 20% hard. Steps 5000+: 10% easy, 40% medium, 50% hard.
3. **Dynamic adjustment**: Every 500 steps, re-evaluate difficulty distribution. Prompts the model has "mastered" (pass rate > 95%) get retired. New harder prompts are introduced.
4. **For GRPO specifically**: Curriculum ensures groups always have a mix of successes and failures. Without curriculum, hard prompts give all-zero groups (useless).

Evidence: DeepSeek-R1 used implicit curriculum — starting with easy math/code problems, the model developed basic reasoning, then solved progressively harder problems without explicit scheduling.
Review: *Chapters 7 and 12 (GRPO; LLM Agentic Training).*

Q20: You have a budget of 64 A100-80GB GPUs and need to RL-train a 70B model. Design the allocation.

Answer: 8 nodes $\times$ 8 GPUs = 64 total. Need to split across generation, scoring, and training.
My allocation:

- **Generation**: 24 GPUs (3 nodes). 6 vLLM instances with TP=4. INT8 weights = 70GB/model, leaves room for KV cache. Continuous batching, batch $\approx$ 128 total.
- **Scoring**: 8 GPUs (1 node). RM (INT8, TP=4) + Reference model (INT8, TP=4) on same node. Or share 4 GPUs with TP=4 alternating between RM and ref.
- **Training**: 32 GPUs (4 nodes). FSDP across all 32. Each GPU holds $\sim$70B/32 = 2.2GB params + optimizer fraction. Plenty of headroom for activations. Gradient checkpointing for safety.

Expected throughput:

- Generation: 6 instances $\times$ $\sim$80 responses/min = 480 responses/min
- Training: Batch of 128, one step every $\sim$15s (training only, no gen wait)
- Overlap: While training step N (15s), generation produces $\sim$120 responses for step $N+1$. Perfect pipeline.

Bottleneck analysis: Generation takes ∼45s for 128 responses. Training takes ∼15s. Scoring takes ∼5s. Generation is bottleneck. Could move 8 GPUs from training to gen (40 gen, 24 training) to balance, but then training is bottleneck. Current allocation is near-optimal.
Alternative if memory-tight: Move scoring onto training nodes (time-share: score during gen, train during training). Saves 8 GPUs. Slightly worse pipelining but works.
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

28.5 GRPO Variants and Advanced RL Questions

Q21: What is DAPO and how does it improve over standard GRPO?

Answer: DAPO (Dynamic Adaptive Policy Optimization) introduces 5 key modifications:
1. Clip-Higher (asymmetric clipping): Standard PPO/GRPO clips both directions equally at $\epsilon = 0.2$. DAPO uses $\epsilon_{\text{low}} = 0.2$ but $\epsilon_{\text{high}} = 0.28$. This allows the model to *increase* good action probabilities more aggressively while still restricting how much it suppresses bad ones. Intuition: exploration needs more room than exploitation.
2. Overlong Filtering: If a response hits the max length limit (truncated, no EOS token), it's masked entirely from the loss. Rationale: truncated responses contain no natural stopping signal — training on them teaches the model that "stopping mid-sentence" is acceptable.
3. Token-level Loss: Loss is normalized by total token count across all sequences, not by number of sequences. This prevents longer sequences from dominating the gradient.
4. Soft Overlong Punishment: Instead of binary truncation filtering, apply a gradual penalty as responses approach max length. $r_{\text{soft}} = -c \cdot \max(0, \text{len} - L_{\text{soft}})/(L_{\max} - L_{\text{soft}})$.
5. Dynamic Sampling: Resample prompts during training to ensure each batch has a mix of success/failure (not yet in TRL).
When to use: Large-scale reasoning RL where you need maximum exploration and long completions (32K+ tokens). The asymmetric clipping is particularly valuable.
Review: *Chapters 7 and 8 (GRPO; Preference Optimization Variants).*

Q22: Explain the vLLM train-inference mismatch. Why does it happen and how do TIS/MIS fix it?

Answer: **The problem**: When using vLLM for generation and a training framework (DeepSpeed/FSDP) for updates, the same model with the same weights produces *different* token probabilities. This happens because:

- Different numerical kernels (vLLM uses custom CUDA kernels optimized for throughput)
- Different attention implementations (Flash Attention in training vs PagedAttention in vLLM)
- Different precision handling (FP8/INT8 in vLLM vs BF16 in training)
- Batching differences affecting layer normalization numerics

This silently breaks PPO's on-policy assumption: we compute the ratio $\pi_\theta/\pi_{\text{old}}$ using π_{old} from vLLM but π_θ from the training framework. The ratio is wrong from step zero!
TIS (Truncated Importance Sampling): Correct the gradient by multiplying by $\min(\pi_{\text{train}}/\pi_{\text{inference}}, C)$. The min with cap C prevents extreme corrections from destabilizing training. Typical $C = 2.0$.
MIS (Masked Importance Sampling): More aggressive — simply discard any token where $\pi_{\text{train}}/\pi_{\text{inference}} > C$. Zero contribution to gradient. Prevents any badly-estimated token from affecting the update.
Sequence vs Token level: Sequence-level IS is theoretically correct (unbiased); token-level IS is biased but lower variance. In practice, sequence-level with truncation works best.
Review: *Chapters 7 and 11 (GRPO; System Architecture).*

Q23: GSPO vs GRPO — what's the fundamental difference and when does it matter?

Answer: **GRPO**: Computes importance ratio *per token*: $w_{i,t} = \pi_\theta(o_{i,t}|q, o_{i,<t}) / \pi_{\text{old}}(o_{i,t}|q, o_{i,<t})$, then clips each token independently.
GSPO: Computes importance ratio at the *sequence level*: $s_i(\theta) = (\pi_\theta(o_i|q)/\pi_{\text{old}}(o_i|q))^{1/|o_i|}$ — the geometric mean of token probabilities. Clips this single sequence-level ratio.
Why it matters: GRPO's per-token clipping treats each token as independent, but in language they're deeply correlated. A small per-token change early in the sequence compounds exponentially over many tokens. GSPO captures this by looking at the full sequence probability.
Length normalization: The $1/|o_i|$ exponent ensures fair comparison across different-length sequences. Without it, longer sequences would always have lower probability ratios.
When to use GSPO: When training goes off-policy (`steps_per_generation > 1` or `num_iterations > 1`). If fully on-policy (ratio ≈ 1), GRPO and GSPO are equivalent.
Review: *Chapters 7 and 8 (GRPO; Preference Optimization Variants).*

Q24: The paper "It Takes Two" shows G=2 matches G=16. How is that possible?

Answer: The key insight is that GRPO's effectiveness doesn't come from accurate advantage estimation (which would need large G), but from an **implicit contrastive objective**.
With $G = 2$ and binary rewards (one correct, one wrong): $\hat{A}_{\text{correct}} = +1$, $\hat{A}_{\text{wrong}} = -1$ (after normalization). The loss becomes: increase probability of correct response, decrease probability of wrong response. This is essentially a DPO-style contrastive loss!
Why large G doesn't help much: The normalized advantage $\hat{A}_i = (r_i - \mu)/\sigma$ already creates a contrast between good and bad. More samples give a better μ estimate, but the gradient direction is dominated by the *contrast* between best and worst, not the mean accuracy.
Compute savings: $G = 2$ means 8× less generation compute than $G = 16$. Since generation is 60% of training time, this translates to ∼4× faster training.
Caveat: Works best when pass rate is 30–70%. If pass rate is very low (<10%), $G = 2$ often gives two failures (no signal). Need larger G for hard problems.
Review: *Chapter 7 (GRPO).*

Q25: What is SAPO and how does its soft gating differ from hard clipping?

Answer: Standard PPO/GRPO uses hard clipping: $\text{clip}(r, 1-\epsilon, 1+\epsilon)$. At the boundary, the gradient suddenly drops to zero. This creates a "dead zone" where the model receives no learning signal.
SAPO replaces this with a smooth sigmoid gate: the gradient is gradually attenuated as the ratio moves away from 1, never suddenly zeroed out. It uses asymmetric temperatures:

- $\tau_+ = 1.0$ for positive advantages (standard attenuation)
- $\tau_- = 1.05$ for negative advantages (slightly more aggressive attenuation for suppression)

Benefits: (1) No "cliff" in gradient landscape. (2) Tokens slightly outside the clip range still contribute (attenuated, not zeroed). (3) More stable optimization trajectory. (4) Sequence-coherent — considers the full sequence context.
Trade-off: Slightly less restrictive trust region than hard clipping, so requires careful temperature tuning. But more robust to hyperparameter choices overall.
Review: *Chapters 7 and 8 (GRPO; Preference Optimization Variants).*

28.6 DPO Extensions Questions

Q26: Compare f-DPO divergence choices. When would you use forward KL vs JS vs reverse KL?

Answer: Standard DPO uses reverse KL implicitly ($D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$):

- **Reverse KL** (default): Mode-seeking. The policy concentrates probability where the reference is high. Avoids generating text the reference wouldn't. Good for safety (conservative).
- **Forward KL**: Mass-covering. The policy tries to cover all modes of the reference, even low-probability ones. Good for diversity but can generate low-quality outputs.
- **Jensen-Shannon**: Symmetric compromise between forward and reverse. Balanced mode-coverage and mode-seeking. Often best for general alignment.
- **Alpha-divergence** ($\alpha = 0.5$): Interpolates between forward ($\alpha = 0$) and reverse ($\alpha = 1$). Tunable.

Practical recommendation: Start with reverse KL (standard DPO). If the model is too conservative (won't try creative solutions), switch to JS divergence. If diversity is critical (creative writing, brainstorming), try forward KL.
Review: *Chapters 6 and 8 (DPO; Preference Optimization Variants).*

Q27: Your DPO preference data has 15% label noise. What do you do?

Answer: Three options in order of sophistication:
1. Robust DPO (best for known noise rate): Analytically debiases the loss: $\mathcal{L}_{\text{robust}} = \frac{(1-\varepsilon)\mathcal{L}_{\text{DPO}}(y_w,y_l) - \varepsilon\mathcal{L}_{\text{DPO}}(y_l,y_w)}{1-2\varepsilon}$. Set $\varepsilon = 0.15$. This provably recovers the clean DPO objective in expectation. TRL: `loss_type="robust", label_smoothing=0.15`.
2. IPO (best when noise rate unknown): Squared loss with target margin. Mislabeled pairs have bounded influence (the squared loss doesn't diverge). More robust to arbitrary noise patterns without needing to know ε. TRL: `loss_type="ipo"`.
3. TR-DPO (best for distribution shift): Updates the reference model via EMA during training. Even if early data is noisy, the evolving reference helps the model self-correct. TRL: `sync_ref_model=True, ref_model_mixup_alpha=0.6`.
Data-side fixes: (1) Filter pairs with <70% inter-annotator agreement. (2) Use RM to score pairs; discard those where RM disagrees with label. (3) Active learning: re-label the most uncertain pairs.
Review: *Chapters 6 and 8 (DPO; Preference Optimization Variants).*

Q28: What is SimPO and why is being reference-free advantageous?

Answer: SimPO uses the average log-probability of a response as an implicit reward signal: $r(x,y) = \frac{1}{|y|}\sum_t \log \pi_\theta(y_t|x, y_{<t})$ — no reference model needed.
The loss adds a target margin γ: the chosen response should have average log-prob at least γ higher than rejected.
Why reference-free matters:

1. **Memory**: No reference model = 70–140GB saved for 70B models. Can train larger models on same hardware.
2. **Simplicity**: No need to manage/load/serve a second model copy.
3. **No stale reference**: DPO's reference becomes increasingly irrelevant as training progresses. SimPO doesn't have this problem.
4. **Length normalization built in**: The $1/|y|$ naturally prevents length bias (DPO needs explicit handling).

Trade-off: Without a reference anchor, the model has more freedom to collapse or drift. The γ margin and length normalization partially mitigate this, but SimPO can be less stable than DPO for aggressive training.
Review: *Chapter 8 (Preference Optimization Variants).*

Q29: Explain Iterative RPO. Why combine DPO with NLL loss for reasoning?

Answer: Standard DPO for reasoning has a subtle failure mode: it learns to *discriminate* (assign higher implicit reward to correct traces) but doesn't necessarily learn to *generate* them.
Why: DPO's gradient pushes chosen probability up and rejected probability down. But the chosen response might be so different from what the model would generate that increasing its probability doesn't teach the model how to produce similar reasoning patterns.
RPO's fix: Add a negative log-likelihood (NLL/SFT) loss on the chosen response: $\mathcal{L} = \mathcal{L}_{\text{DPO}} + \alpha \cdot \mathcal{L}_{\text{NLL}}(y_w)$.
The NLL term explicitly trains the model to generate the winning response step by step. The DPO term ensures it also learns to avoid the losing response. Combined: the model learns both "how to reason correctly" (NLL) and "what to avoid" (DPO).
Iterative: Generate responses → check correctness → create pairs → train with RPO → repeat. Each iteration the model gets better at generating correct reasoning, creating higher-quality training data for the next round.
TRL: `loss_type=["sigmoid", "sft"], loss_weights=[1.0, 1.0]`
Review: *Chapters 8 and 13 (Preference Optimization Variants; RL for Large Reasoning Models).*

28.7 GPU Architecture and Hardware Questions

Q30: Explain the GPU memory hierarchy. Why does it matter for LLM inference?

Answer: From fastest to slowest:

1. **Registers**: Per-thread, ∼256 KB/SM. Instant access (0 cycles latency).
2. **SRAM (Shared Memory)**: Per-SM, ∼192–228 KB/SM (A100: 164 KB configurable). Bandwidth: ∼19 TB/s aggregate. Latency: ∼20 cycles.
3. **L2 Cache**: Shared across GPU, 40–60 MB (H100: 50 MB). Bandwidth: ∼5 TB/s. Latency: ∼200 cycles.
4. **HBM**: Main GPU memory, 80 GB (A100). Bandwidth: 2–3.35 TB/s. Latency: ∼400 cycles.
5. **CPU DRAM**: Via PCIe, 512 GB+. Bandwidth: 32–64 GB/s. Latency: ∼10K cycles.

Why it matters for LLMs: Autoregressive generation reads the full model weights (∼140 GB for 70B) for every single token. At 2 TB/s HBM bandwidth, that's 70ms just to stream the weights. The actual computation (one matrix-vector multiply) takes only 0.5ms. The GPU is 99% waiting for data.
Flash Attention exploits this: By keeping intermediate results (QK scores, softmax) in SRAM (19 TB/s) instead of writing them to HBM (2 TB/s), it eliminates 90% of the memory traffic for attention. The compute is the same, but HBM reads/writes drop 10×.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q31: How does Flash Attention work? What is the online softmax trick?

Answer: **Problem**: Standard attention materializes the $n \times n$ attention matrix in HBM. For $n = 8192$: $8192^2 \times 2 = 134$ MB per head, 4.3 GB per layer with 32 heads. Must write to HBM then read back for softmax and multiply — 3 full HBM round-trips.
Flash Attention solution: Never store the full $n \times n$ matrix. Process in tiles that fit in SRAM.
Algorithm:

1. Split Q into blocks of size B_r rows, K/V into blocks of B_c rows.
2. For each Q block: iterate over all K blocks, computing partial attention scores.
3. **Online softmax trick**: Maintain running max m and running sum ℓ for softmax normalization. When processing a new K block, update: $m_{\text{new}} = \max(m_{\text{old}}, \max(\text{scores}))$, rescale previous accumulator by $e^{m_{\text{old}} - m_{\text{new}}}$, add new contribution.
4. Output is accumulated incrementally — never needs the full $n \times n$ matrix.

Key insight: Softmax is normally a global operation (max and $\sum$ over all elements). The online trick decomposes it into local updates with a correction factor. Mathematically exact — no approximation.
Result: Memory $O(n)$ instead of $O(n^2)$. Speed 2–4× faster (fewer HBM accesses, more time in SRAM).
Flash Attention 2: Better work partitioning across warps, reduces non-matmul FLOPs by 2×.
Flash Attention 3 (H100/Hopper): Uses Tensor Memory Accelerator (TMA) for async loads, warp specialization (producer/consumer warps), FP8 support.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q32: Explain PagedAttention. How does it solve the KV cache problem?

Answer: **The problem**: During generation, each sequence needs a KV cache (stores K and V tensors for all previous tokens). For a 70B model: each token needs $2 \times n_{\text{layers}} \times d_{\text{model}} \times 2$ bytes $= 2 \times 80 \times 8192 \times 2 \approx 2.5$ MB per token. A 2048-token sequence: ∼5 GB of KV cache.
Traditional allocation: Pre-allocate max_sequence_length for each active sequence. If max=2048 but average=500, you waste 75% of allocated memory. With 50 concurrent sequences, that's hundreds of GB wasted.
PagedAttention: Inspired by OS virtual memory:

1. KV cache is split into fixed-size *blocks* (pages), each holding KV for 16 tokens.
2. A *block table* (like a page table) maps logical token positions to physical memory blocks.
3. Blocks are allocated on demand as the sequence grows. No pre-allocation waste.
4. Freed blocks return to a pool immediately when sequences finish.

Extra benefits:

- **Prefix sharing**: Multiple sequences with the same system prompt share KV cache blocks (copy-on-write). Saves 30–50% memory for chat applications.
- **Preemption**: Can "swap out" a low-priority sequence's blocks to CPU, freeing GPU memory for higher-priority requests.
- **Near-zero fragmentation**: Internal fragmentation limited to last block (<16 tokens). External fragmentation eliminated (any free block can be used anywhere).

Result: 3–5× more concurrent sequences in the same memory → 3–5× better throughput for serving.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q33: Compare NVLink vs InfiniBand. When do you use each in RLHF training?

Answer:
NVLink (intra-node, GPU-to-GPU):

- Bandwidth: 600 GB/s (A100), 900 GB/s (H100) — total bidirectional

- Latency: ∼1 μs
- Scope: Within one physical node (8 GPUs connected via NVSwitch)
- Use case: **Tensor Parallelism** (TP=8). Each layer's matrix multiply is split across GPUs, requiring AllReduce after every layer. Needs ultra-high bandwidth + low latency.

InfiniBand NDR (inter-node, node-to-node):

- Bandwidth: 400 Gb/s = 50 GB/s per port. With 8 ports (GPUDirect RDMA): 400 GB/s aggregate per node.
- Latency: ∼1–5 μs (RDMA)
- Scope: Between nodes in a cluster. Requires switches (fat-tree topology).
- Use case: **Data Parallelism / FSDP** gradient synchronization. AllReduce of gradients happens once per training step (not per layer), so latency tolerance is higher.

In RLHF specifically:

- *Generation*: TP=8 over NVLink within node. Multiple vLLM instances across nodes don't communicate (embarrassingly parallel).
- *Training*: TP=8 over NVLink intra-node + FSDP over InfiniBand inter-node. Gradients synced after full backward pass.
- *Weight sync*: Training → Generation uses InfiniBand (140 GB transfer, async, takes ∼3s at 50 GB/s).

Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

28.8 Optimization and Training Questions

Q34: Explain Adam vs AdamW. Why does the difference matter for LLMs?

Answer: **Adam with L2 regularization**: $\theta_{t+1} = \theta_t - \alpha \cdot (\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon) + \lambda\theta_t)$. The weight decay term $\lambda\theta_t$ is *inside* the adaptive scaling. Parameters with large gradients (large v_t) get *less* weight decay (divided by $\sqrt{v_t}$). This is not true weight decay — it's scale-dependent.
AdamW (decoupled weight decay): $\theta_{t+1} = (1 - \alpha\lambda)\theta_t - \alpha \cdot \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$. Weight decay is applied *outside* and *before* the adaptive update. Every parameter gets the same proportional decay regardless of gradient history.
Why it matters for LLMs:

1. LLMs have parameters spanning many orders of magnitude (embedding layers vs attention vs FFN). Adam's coupled L2 effectively penalizes small-gradient params more than large-gradient ones — wrong behavior.
2. Decoupled WD provides uniform regularization across all layers, preventing some layers from growing unbounded while others over-shrink.
3. Empirically: AdamW gives 2–5% better perplexity on long pretraining runs compared to Adam+L2 with the same effective regularization.

For RL specifically: Often use $\lambda = 0$ (no weight decay). The KL penalty provides regularization. But for SFT: $\lambda = 0.01$–0.1 with AdamW is standard.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q35: Why is learning rate warmup necessary? What happens without it?

Answer: **The problem**: Adam's second moment estimate $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ starts at $v_0 = 0$. The bias correction $\hat{v}_t = v_t/(1-\beta_2^t)$ compensates mathematically, but in practice:

- First few steps: v_t is based on 1–5 gradient samples. Highly inaccurate estimate of true variance.
- If a parameter happens to get a small gradient initially, v_t is tiny → effective LR is huge → catastrophic update.
- The bias correction amplifies early updates: at step 1, $\hat{v}_1 = v_1/(1-0.999) = 1000 \cdot v_1$.

Without warmup: First 10–100 steps often have gradient spikes that permanently damage the model. Early representations get scrambled before the optimizer stabilizes.
Warmup fix: Start with LR ≈ 0 and linearly increase to target over W steps (typically 3–10% of training). By the time LR reaches full value, v_t has accumulated enough samples to be accurate.
Typical settings:

- Pretraining: 2000 steps warmup (∼1% of 200K steps)
- SFT: 100 steps warmup (∼5% of 2000 steps)
- RL (PPO/GRPO): 20–50 steps warmup (short, model already stable from SFT)

Review: *Chapters 1 and 10 (LLM Architecture; SFT Best Practices).*

Q36: Compare learning rate schedules. Which would you choose for RL fine-tuning?

Answer:
Cosine decay: $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$. Standard for pretraining and SFT. Smooth decay, most time at moderate LR.
Linear decay: $\eta_t = \eta_{\max}(1 - t/T)$. Simpler, similar results to cosine for short runs.
WSD (Warmup-Stable-Decay): Warmup → constant LR for 80% → rapid decay in last 20%. New standard for pretraining. The "stable" phase gives consistent learning; the final decay squeezes out remaining gains.
Constant: No decay. $\eta_t = \eta_{\max}$ after warmup.
For RL fine-tuning (PPO/GRPO), I'd choose: Constant with short warmup. Reasons:

1. RL training length is highly unpredictable (you stop based on win-rate, not epochs).
2. Cosine/linear decay assumes you know the total steps in advance.
3. The LR is already very low (10^{-6}), further decay makes updates invisible.
4. PPO's adaptive KL controller already modulates the effective step size.
5. If you must decay: use linear decay over a generous budget, stop early when metrics plateau.

Review: *Chapters 1 and 5 (LLM Architecture; PPO).*

Q37: Why is gradient clipping critical for RL training but less important for SFT?

Answer: **SFT**: Supervised loss is smooth and well-behaved. Gradient norms are consistent across batches (typically 0.1–1.0). Clipping at 1.0 rarely activates — it's a safety net.
RL (PPO/GRPO): Gradient norms are highly variable because:

1. **Reward variance**: One batch might have all high-reward responses, next might have all low. The advantage $\hat{A}$ swings wildly.
2. **Ratio explosion**: If a rare token's probability changed a lot, $r_t = \pi_{\text{new}}/\pi_{\text{old}}$ can be very

large → large gradient before clipping kicks in.

3. **Sparse reward**: In GRPO with binary rewards, some prompts give all-correct (advantage ≈ 0) then suddenly a hard prompt gives extreme advantages.

4. **KL term**: The KL penalty gradient can spike when policy diverges.

Without clipping: A single bad batch can produce a gradient 100× normal magnitude → destroys the model in one step. Recovery is impossible (catastrophic forgetting of all pretraining).
Typical setting: `max_grad_norm=1.0`. Some use 0.5 for extra safety in early RL training. The norm is computed globally across all parameters (not per-layer).
Monitoring: If clipping activates more than 20% of steps, your LR is probably too high or your batch size too small.
Review: *Chapters 5 and 7 (PPO; GRPO).*

Q38: BF16 vs FP16 for training. When does the choice matter?

Answer:
FP16: 1 sign + 5 exponent + 10 mantissa bits. Range: ±65504. Precision: ∼3.3 decimal digits.
BF16: 1 sign + 8 exponent + 7 mantissa bits. Range: $\pm 3.4 \times 10^{38}$ (same as FP32!). Precision: ∼2.4 decimal digits.
Why BF16 wins for LLMs:

1. **No loss scaling needed**: FP16's small range (±65K) means gradients and activations frequently overflow/underflow. Requires dynamic loss scaling (multiply loss by 1024, divide gradients back). BF16 has FP32's range — overflow is essentially impossible.

2. **Simpler code**: No loss scaler, no inf/nan checking, no dynamic scaling adjustment.

3. **Critical for RL**: RL gradients are noisier and spikier than SFT. FP16 loss scaling often fails (picks wrong scale, causes NaN). BF16 "just works."

When FP16 might be better: If you need maximum precision (some scientific computing tasks) and can manage the loss scaling. FP16 has 3 more mantissa bits = slightly more accurate results.
FP32 master weights: Even with BF16 forward/backward, accumulate gradient updates in FP32 to prevent rounding errors from compounding over thousands of small steps. Standard practice for all LLM training.
Review: *Chapter 2 (Systems Foundations for LLMs).*

28.9 Reward Model and SFT Questions

Q39: Derive the Bradley-Terry reward model loss. What are its limitations?

Answer: **Bradley-Terry model**: Given two responses, the probability the better one (y_w) is preferred: $P(y_w \succ y_l|x) = \sigma(r(x, y_w) - r(x, y_l))$ where σ is the sigmoid.
MLE derivation: Given N preference pairs, maximize likelihood: $\prod_i P(y_w^i \succ y_l^i)$. Take negative log: $\mathcal{L} = -\sum_i \log \sigma(r(x_i, y_w^i) - r(x_i, y_l^i))$.
Limitations:

1. **No ties**: BT can't model "equally good" — forces a strict preference.

2. **Transitivity**: Assumes if A>B and B>C then A>C. Humans aren't transitive.

3. **Context-free**: Same reward regardless of what alternatives were available.

4. **Scalar collapse**: Compresses all quality dimensions into one number. A response can be safe but unhelpful — RM must trade off.

5. **Length bias**: Longer responses get higher scores (more information = more likely to contain what annotator wanted). Must explicitly decorrelate.

Mitigations: Margin loss (require minimum gap δ), reward centering (subtract running mean), length penalty during training, multi-head RM (separate scores for helpfulness/safety/accuracy).
Review: *Chapter 9 (Reward Model Training).*

Q40: What is sequence packing in SFT and why does it matter?

Answer: **Problem**: Training examples have variable length. Standard batching pads all examples to max_length. If max=4096 but average=500, you waste 88% of compute on padding tokens (which contribute zero gradient).
Packing solution: Concatenate multiple short examples into a single max_length sequence. Separate with EOS tokens. Train on all examples simultaneously.
Example: Instead of 4 sequences padded to 4096 (16K tokens, 14K padding), pack into 1 sequence of 4096 with 4 examples end-to-end (4096 real tokens, 0 padding). **4× more efficient.**
Critical detail — block-diagonal attention mask: Without special handling, example 2 attends to example 1's tokens (cross-contamination). Must use a block-diagonal attention mask that restricts each example to only attend to its own tokens.
In TRL: `SFTConfig(packing=True, max_seq_length=4096)`. Handles mask automatically.
Caveats: (1) Longer examples still need their own batch entries (can't split mid-sequence). (2) Slight implementation complexity for position embeddings (reset per example). (3) Some argue packing changes the effective batch size (more examples per step) — adjust LR accordingly.
Review: *Chapter 10 (SFT Best Practices and Techniques).*

Q41: Explain completion-only masking for SFT. What happens if you don't use it?

Answer: In chat-format SFT data: `[system] + [user message] + [assistant response]`. Standard NLL loss computes loss on all tokens including the system prompt and user message.
Problem without masking: The model wastes capacity learning to predict the user's message (which it will never need to generate at inference). Worse: if the training data has diverse user messages, the model gets confused about "whose turn is it?"
Completion-only masking: Set loss weight to 0 for all tokens in the prompt (system + user). Only compute loss on assistant response tokens.
TRL: `DataCollatorForCompletionOnlyLM(response_template="<|assistant|>")`
Impact: Typically 5–15% better on instruction-following benchmarks. Faster convergence (gradient signal is concentrated on useful tokens). No change to compute cost.
Subtlety: Must include the response template token in the loss (teaches the model to start responding). But exclude everything before it.
Review: *Chapter 10 (SFT Best Practices and Techniques).*

Q42: How does SFT quality affect the RL ceiling? What is the pass@k diagnostic?

Answer: **Ceiling theorem (informal)**: RL can only reinforce behaviors the model can already produce with non-negligible probability. If the SFT model has 0% chance of generating a correct solution, RL will never find it.
Why: GRPO/PPO sample from the current policy and reinforce good samples. If good samples don't exist in the distribution, there's nothing to reinforce. RL is exploration-limited by the base policy's support.
pass@k diagnostic: Generate k responses per prompt, check if *any* is correct:

- pass@1: Model's typical performance (greedy/low temp).
- pass@8: Upper bound of what GRPO with $G = 8$ can achieve.
- pass@64: Upper bound for aggressive Best-of-N.
- pass@256: Approximate ceiling for RL improvement.

Interpretation:

- pass@1=20%, pass@64=80%: Great! 4× headroom for RL. Strong gains expected.
- pass@1=20%, pass@64=25%: Almost no headroom. RL won't help much. Need better SFT first.
- pass@1=5%, pass@64=60%: Model *can* solve it but rarely does. Perfect case for RL (reinforce the rare successes).

Rule: If pass@64 $< 1.5\times$ pass@1, invest in better SFT data before starting RL.
Review: *Chapters 7 and 10 (GRPO; SFT Best Practices).*

Q43: Design a multi-objective reward system for a chat model. How do you balance helpfulness vs safety?

Answer: **Architecture**: Separate reward models for each objective:

- r_{helpful}: Trained on helpfulness preferences (quality, accuracy, completeness)
- r_{safe}: Trained on safety preferences (refusals, harmlessness, no hallucination)
- r_{format}: Rule-based (follows instructions, proper formatting, appropriate length)

Combination strategies:

1. **Weighted sum** (simplest): $r = w_1 r_{\text{helpful}} + w_2 r_{\text{safe}} + w_3 r_{\text{format}}$. Problem: safety can be outweighed by helpfulness.
2. **Constrained** (safer): Maximize r_{helpful} subject to $r_{\text{safe}} > \tau$. Implemented via: $r = r_{\text{helpful}} - \lambda \cdot \max(0, \tau - r_{\text{safe}})$ with large λ.
3. **GDPO normalization** (best for GRPO): Normalize each reward independently within group, then combine: $\hat{A} = w_1 \hat{A}_{\text{helpful}} + w_2 \hat{A}_{\text{safe}}$. Prevents one reward from dominating due to scale differences.
4. **Lexicographic**: Safety is hard constraint (must pass), then optimize helpfulness. Train in stages: safety alignment first, then helpfulness.

Practical weights: Start with $w_{\text{safe}} = 2.0, w_{\text{helpful}} = 1.0, w_{\text{format}} = 0.5$. Safety gets $2\times$ weight because its failure mode (harmful content) is much worse than helpfulness failure (mediocre answer).
Review: *Chapters 9 and 12 (Reward Model Training; LLM Agentic Training).*

28.10 System Architecture Extension Questions

Q44: How does speculative decoding work? When does it help for RLHF?

Answer: **Problem**: Large model generates one token per forward pass (∼70ms for 70B). Slow.
Speculative decoding:

1. **Draft**: Small model (1–7B) generates k candidate tokens quickly (∼5ms for all k).
2. **Verify**: Large model does one forward pass scoring all k tokens in parallel. Accepts tokens where $p_{\text{large}}(t_i) \geq p_{\text{draft}}(t_i)$ (always). Probabilistically accepts others.
3. **Result**: On average, 3–4 tokens accepted per verification step. Speedup: 2–3×.

Key property: The output distribution is *identical* to sampling from the large model alone. No quality loss. The draft model only affects speed, not output.

For RLHF specifically: Generation is 60% of compute. 2–3× speedup on generation = 1.5–2× end-to-end speedup. Combined with vLLM + INT8: generation goes from the bottleneck to parity with training.
Limitations: (1) Draft model must share tokenizer. (2) Less effective at high temperature (draft model less accurate). (3) Needs additional GPU memory for draft model. (4) Diminishing returns beyond $k = 5$ (acceptance rate drops).
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

Q45: Explain the roofline model. How do you determine if a kernel is compute-bound or memory-bound?

Answer: The roofline model plots achievable performance (FLOPS) as a function of arithmetic intensity (FLOPS per byte of memory traffic).
Two regimes:

- **Memory-bound** (left of crossover): Performance limited by how fast you can feed data to compute units. Actual FLOPS = bandwidth × arithmetic intensity. GPU utilization < 100%.
- **Compute-bound** (right of crossover): Performance limited by peak FLOPS. Memory is fast enough. GPU at max utilization.

Crossover point: Peak FLOPS / Peak Bandwidth. For A100: 312 TF / 2 TB/s = 156 FLOP/byte.
LLM operations:

- **Autoregressive generation** (batch=1): Read 140GB weights, do 140G FLOPs = 1 FLOP/byte. *Extremely* memory-bound (156× below crossover). Only 0.6% GPU utilization.
- **Training forward pass** (batch=128, seq=2048): Arithmetic intensity ≈ 200+ FLOP/byte. Compute-bound. Near peak utilization.
- **Attention** (long sequence): $O(n^2 d)$ FLOPs / $O(n^2 + nd)$ bytes. For long n: compute-bound. For short n: memory-bound. Flash Attention keeps it in SRAM regardless.

Practical use: If your kernel is memory-bound, reduce memory traffic (quantization, caching, tiling). If compute-bound, reduce FLOPs (pruning, distillation, lower precision).
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q46: How does continuous batching work and why is it essential for RLHF generation?

Answer: **Static batching**: Start B sequences. Wait for ALL to finish. If one sequence generates 500 tokens and another generates 50 tokens, the 50-token sequence's GPU slot sits idle for 450 tokens.
Continuous batching (iteration-level scheduling): After each generation step, check which sequences are done. Immediately insert new sequences into freed slots. GPU slots are never idle.
Why essential for RLHF:

1. RLHF generates diverse outputs (high temperature). Length variance is huge — some responses are 50 tokens, others 2000+.
2. Without continuous batching: average utilization ∼40–50% (waiting for slowest sequence).
3. With continuous batching: utilization >90%. Throughput 2–3× higher.
4. RLHF needs large batches (128+ responses per step). Generating 128 responses with static batching requires max_tokens × 128 sequential steps. Continuous batching amortizes this.

Implementation: vLLM's scheduler checks after every decode step. Preemption: if a new high-priority request arrives and memory is full, can swap out a low-priority sequence's KV cache to CPU and resume later.
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

28.11 Transformer Architecture Questions

Q: Why does RoPE dominate over learned absolute positional embeddings in modern LLMs?

Answer: RoPE encodes *relative* position directly into the Q/K dot product via rotation matrices. Key advantages:

1. Attention scores depend only on relative distance $i-j$, not absolute position — this generalizes better to unseen sequence lengths.
2. Can be extended beyond training length via frequency scaling (NTK-aware, YaRN) without retraining.
3. No additional parameters (rotations are deterministic from position index).
4. Learned absolute embeddings are fixed to training length and don't extrapolate — a model trained at 4K context fails at 8K.

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: Explain SwiGLU and why it replaced ReLU in modern transformers.

Answer: SwiGLU: $\text{FFN}(x) = W_2(\text{Swish}(W_1 x) \odot W_3 x)$, where $\text{Swish}(x) = x \cdot \sigma(x)$.
Why it's better:

- The *gating* mechanism ($\odot W_3 x$) allows the network to selectively suppress or amplify dimensions — more expressive than pointwise ReLU.
- Swish is smooth (no dead neurons like ReLU's zero-gradient region).
- Empirically: 1–2% improvement on language modeling benchmarks at same FLOP count.
- Tradeoff: requires 3 weight matrices instead of 2 (solved by reducing hidden dim from $4d$ to $8d/3$).

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: What is Grouped Query Attention (GQA) and why does Llama-3 use it?

Answer: Standard MHA: H query heads, H key heads, H value heads. GQA: H query heads but only $G < H$ key/value heads (shared across query groups).
Llama-3 70B: 64 query heads, 8 KV heads (each KV head shared by 8 query heads).
Benefits:

- KV cache size reduced by $H/G = 8\times$ — critical for inference (KV cache is the dominant memory cost at long sequences).
- Minimal quality loss (<0.5% on benchmarks) because KV patterns are highly correlated across heads.
- Inference throughput increases proportionally to KV cache reduction (more sequences fit in memory = higher batch size).

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: Why did decoder-only architectures win over encoder-decoder for LLMs?

Answer:

1. **Unified objective**: Pretraining = fine-tuning = inference all use next-token prediction. No architectural mismatch.
2. **Parameter efficiency**: All parameters contribute to generation. In encoder-decoder, encoder params are "wasted" during pure generation tasks.
3. **Simpler scaling**: One model, one loss function, one set of hyperparameters to tune.
4. **KV cache efficiency**: Decoder-only has one KV cache; encoder-decoder has two (encoder + decoder cross-attention).
5. **Emergent few-shot**: Decoder-only naturally supports in-context learning (prepend examples to the prompt).

Encoder-decoder still wins for seq2seq tasks with fixed input length (translation), but these are a shrinking fraction of LLM use cases.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

28.12 Flash Attention Questions

Q: Flash Attention computes the same result as standard attention but is 2–4× faster. How is this possible if it does the same number of FLOPs?

Answer: Flash Attention is faster because it reduces *HBM memory traffic*, not FLOPs. Standard attention materializes the $n \times n$ attention matrix in HBM (slow), reads it back for softmax, reads again for PV multiply — 4 HBM round-trips over $O(n^2)$ data.
Flash Attention tiles the computation so the $n \times n$ matrix is computed and consumed entirely in SRAM (fast, 19 TB/s) without ever writing it to HBM (2 TB/s). The "online softmax" trick enables this by maintaining running statistics.
Result: HBM traffic drops from $O(n^2 d)$ to $O(n^2 d/M)$ where M is SRAM size. Same FLOPs, 10–50× less memory traffic → 2–4× wall-clock speedup.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q: Why doesn't Flash Attention help the FFN layers?

Answer: FFN layers are *compute-bound*, not memory-bound. Their arithmetic intensity ($I \approx 300$ FLOP/byte for large batch GEMMs) is already above the roofline ridge point (156 FLOP/byte on A100).
Flash Attention helps attention because attention is deeply memory-bound ($I \approx 1$–60 FLOP/byte). By keeping data in SRAM, it removes the memory bottleneck.
For FFN: the bottleneck is already the Tensor Cores (not memory bandwidth), so reducing memory traffic doesn't help. Instead, FFN benefits from quantization (reduces weight size → higher arithmetic intensity) and larger batch sizes.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q: Explain the online softmax trick and why it's essential for Flash Attention.

Answer: Standard softmax needs the global maximum $m = \max_j x_j$ before computing any output — this requires seeing all n attention scores first, forcing materialization of the full $n \times n$ matrix. The online softmax trick processes blocks sequentially, maintaining a running (m, ℓ, O) state:

1. Process new block → update running max: $m_{\text{new}} = \max(m_{\text{old}}, \max(s_{\text{new}}))$
2. Rescale old sum: $\ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \cdot \ell_{\text{old}} + \text{new terms}$

3. Rescale output: $O_{\text{new}} = \text{rescaled}(O_{\text{old}}) + \text{new contribution}$

This is mathematically exact — no approximation. It enables block-by-block processing where each block fits in SRAM, never needing the full $n \times n$ matrix in memory.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

28.13 LoRA and PEFT Questions

Q: Why does LoRA work? What theoretical insight justifies low-rank updates?

Answer: Aghajanyan et al. [3] showed that fine-tuning operates in a very low *intrinsic dimensionality* — the effective parameter space for a fine-tuning task is far smaller than the model's total parameter count. A 175B model may have intrinsic dimensionality <10,000 for a given task. LoRA directly exploits this: by constraining updates to rank r ($W' = W + BA$, $B \in \mathbb{R}^{d \times r}$), it restricts learning to an r-dimensional subspace per weight matrix. Since the true task subspace is low-dimensional, this loses almost nothing while reducing trainable parameters by 100–1000×.
Intuition: Fine-tuning doesn't change what the model "knows" (the full-rank W stays frozen); it only adjusts *how* existing knowledge is combined for the new task — a low-rank perturbation.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: Compare QLoRA vs. full LoRA vs. full fine-tuning for a 70B model. When would you choose each?

Answer:

Method	Memory	GPUs	Quality	Use When
Full fine-tune	560+ GB	8+ A100	Best	Unlimited budget; pre-training continuation
LoRA ($r = 16$)	145 GB	2 A100	95–98%	Good budget; general fine-tuning
QLoRA ($r = 16$)	44 GB	1×48GB	93–96%	Single-GPU; prototyping; constrained resources

Decision tree: (1) If task requires deep knowledge change → full fine-tune. (2) If adapting to new style/format → LoRA. (3) If memory-constrained or rapid iteration → QLoRA. (4) If rank matters: start $r = 16$; increase if training loss plateaus above full fine-tune level.
Review: *Chapters 1 and 10 (LLM Architecture; SFT Best Practices).*

Q: What is DoRA and why does it outperform standard LoRA?

Answer: DoRA (Weight-Decomposed Low-Rank Adaptation) decomposes W into magnitude $\|W\|$ and direction $W/\|W\|$, then applies LoRA only to the direction component:

$$W' = m \odot \frac{W + BA}{\|W + BA\|}$$

where m (magnitude) is also trainable but as a simple scalar per output neuron.
Why it helps: Full fine-tuning naturally updates both magnitude and direction independently. Standard LoRA couples them (the low-rank update changes both simultaneously in a constrained way). DoRA decouples them, giving LoRA the same "degrees of freedom" structure as full fine-tuning. Result: 1–3% improvement on reasoning tasks with no extra compute at inference (merge adapters).
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

28.14 Model Compression Questions

Q: Explain AWQ. Why does protecting 1% of weights preserve 99% of quality?

Answer: AWQ (Activation-Aware Weight Quantization) observes that weight importance is highly non-uniform: weights that multiply large activations contribute disproportionately to the output. The key insight: $\|W \cdot X\|$ depends on both W and X. A small weight multiplied by a large activation matters more than a large weight multiplied by a near-zero activation.
AWQ identifies the top 1% of "salient" channels (those with consistently large activation magnitudes across calibration data) and protects them by scaling: multiply salient channels by a factor $s > 1$ before quantization (then divide by s in the activation). This reduces relative quantization error for important channels.
Result: 4-bit quantization with <1% quality loss on 70B models, because the 99% of non-salient weights can tolerate aggressive quantization.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q: When should you use FP8 vs. 4-bit quantization vs. BF16?

Answer:

- **BF16**: Training (policy model in RLHF), when precision matters. Default for any model being updated by gradients.
- **FP8 (E4M3)**: H100 training with Transformer Engine (2× throughput, <0.5% quality loss). Also for inference on H100 when you need maximum throughput.
- **INT8/FP8 inference**: Frozen models in RLHF (reference model, reward model) — not being trained, so reduced precision is safe.
- **4-bit (AWQ/GPTQ)**: Inference serving at scale. Best memory/quality tradeoff for deployment. Also for QLoRA base model.
- **2-bit**: Experimental; edge deployment where memory is extreme constraint. Quality loss 5–10%.

Rule: quantize as aggressively as possible for inference, keep BF16 (or FP8 on H100) for training.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q: Explain NVIDIA 2:4 structured sparsity. What's the speedup and constraint?

Answer: 2:4 sparsity means: in every group of 4 consecutive elements, exactly 2 must be zero. This is enforced at the weight level.
Hardware support: A100/H100 Tensor Cores have dedicated 2:4 sparse GEMM instructions that skip the zero elements, achieving exactly **2× throughput** with no software overhead.
Constraint: You must achieve *exactly* 50% sparsity in this specific pattern. You can't have 30% or 70% sparsity; you can't have arbitrary sparsity patterns. The pruning must respect the 4-element group structure.
How to achieve it: After training (or during fine-tuning), for each group of 4 weights, zero out the 2 smallest by magnitude. Then fine-tune for a few hundred steps to recover quality. Quality loss: typically <1% for large models (70B+).
Review: *Chapter 2 (Systems Foundations for LLMs).*

28.15 Mixture of Experts Questions

Q: Mixtral 8x7B has 47B total parameters but only 13B active per token. Explain how this works and why it's efficient.

Answer: Mixtral replaces each FFN layer with 8 parallel expert FFNs (each ∼7B params for the FFN portion). A router network selects the Top-2 experts per token.
Why 47B total: Attention layers are shared (not replicated) = ∼5B. FFN experts: 8 × ∼5.25B = 42B. Total: ∼47B.
Why 13B active: Per token, only 2 experts fire. Active params = attention (∼5B) + 2 FFN experts (∼2 × 5.25B) ≈ 13B.
Why efficient: Compute cost scales with *active* params (13B), matching a 13B dense model. But capacity (knowledge stored) scales with *total* params (47B), matching much larger models. Result: Mixtral matches Llama-2 70B quality at 13B compute cost.
Memory cost: Still need all 47B params in memory (all experts loaded), so memory = 47B model, but compute = 13B model.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: What is the load balancing problem in MoE and how is it solved?

Answer: Without constraints, the router tends to send most tokens to 1–2 "popular" experts (rich-get-richer dynamics). This causes:

- Capacity waste: 6 of 8 experts are unused, model effectively shrinks to 2-expert size.
- Compute imbalance: If each expert is on a different GPU, popular experts become bottlenecks while others idle.

Solution: Auxiliary load-balancing loss: $\mathcal{L}_{\text{bal}} = \alpha \cdot N \sum_{i=1}^{N} f_i \cdot p_i$, where f_i = fraction of tokens routed to expert i, p_i = mean router probability for expert i. This penalizes uneven distributions.
Alternative: Expert capacity factor — hard cap on max tokens per expert per batch. Overflow tokens are dropped or re-routed.
Typical α: 0.01–0.1 (small enough not to hurt main loss, large enough to prevent collapse).
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

28.16 Diversity in Training Questions

Q: What happens if all N responses in a GRPO group are identical?

Answer: If all N responses are identical: all rewards r_i are equal, so $\sigma_G = 0$ and advantages $\hat{A}_i = (r_i - \mu_G)/\sigma_G$ are undefined (division by zero). In practice, implementations set $\hat{A}_i = 0$ for all, meaning **zero learning signal** — the step is wasted.
Prevention:

1. **Temperature**: Use $\tau = 0.7$–1.0 during generation (not greedy).
2. **Large N**: $N = 8$–16 increases probability of diverse responses.
3. **Duplicate rejection**: DAPO's approach — reject duplicate responses and resample.
4. **Frequency penalty**: Penalize repeated n-grams during generation.
5. **Monitor**: Track unique-response ratio per group. If <50%, increase temperature.

Review: *Chapter 7 (GRPO).*

Q: Explain the diversity-quality tradeoff in RLHF. How do you detect mode collapse?

Answer: **Tradeoff**: High diversity (high entropy/temperature) = varied but potentially random/low-quality responses. Low diversity = consistent but repetitive, reward-hacked responses. **Detecting mode collapse** (all should be monitored during training):

1. **Response entropy**: Compute per-token entropy $H = -\sum p_i \log p_i$. If dropping rapidly → collapse.
2. **Unique n-gram ratio**: Fraction of unique 4-grams across responses to same prompt. Healthy: >0.6.
3. **Reward distribution width**: If σ(rewards) shrinks to near-zero → all responses are the same quality → likely identical.
4. **KL divergence**: If $D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$ is growing rapidly, the policy is moving far from reference → often toward a narrow mode.
5. **Length histogram**: If all responses converge to same length → template behavior.

Fix: Increase KL coefficient β, increase entropy bonus, increase sampling temperature, or rollback to earlier checkpoint.
Review: *Chapters 7 and 9 (GRPO; Reward Model Training).*

28.17 Speculative Decoding Questions

Q: Speculative decoding claims "no quality loss." How can generating tokens differently produce identical output distribution?

Answer: The acceptance/rejection scheme guarantees distributional equivalence:
For each draft token $\hat{x}$ with draft probability $q(\hat{x})$ and target probability $p(\hat{x})$:

- Accept with probability $\min(1, p(\hat{x})/q(\hat{x}))$
- On rejection: sample from the *residual distribution* $\propto \max(0, p(x) - q(x))$

This is mathematically equivalent to sampling directly from p (the target). Proof sketch: the probability of outputting token x is $q(x) \cdot \min(1, p(x)/q(x)) + P(\text{reject}) \cdot \frac{\max(0,p(x)-q(x))}{\sum_y \max(0,p(y)-q(y))} = p(x)$.
The speedup comes from amortizing: when the draft is good (high acceptance), multiple tokens are confirmed in one target forward pass. The guarantee holds regardless of draft quality — bad drafts just give lower speedup (more rejections), not worse quality.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q: Compare Medusa vs. Eagle for speculative decoding. When would you choose each?

Answer:
Medusa: Adds k parallel prediction heads to the target model. Each head independently predicts token at position $t + i$. *Pro*: No separate model, <1% memory overhead. *Con*: Heads predict independently — cannot condition position $t + 2$ on what was predicted at $t + 1$. Acceptance rate: 60–80%.
Eagle: Lightweight autoregressive decoder on target model's hidden states. Draft tokens are generated autoregressively (each conditioned on previous). *Pro*: Captures inter-token dependencies → 85–95% acceptance rate. *Con*: Slightly more memory (small decoder) and sequential draft generation.
Choose Medusa when: Memory is extremely tight; simple integration; moderate speedup is sufficient (2–2.5×).

Choose Eagle when: Maximum speedup needed (3–4×); can afford small extra model; latency-critical single-stream generation.
Choose N-gram when: Repetitive outputs (code, structured data); zero cost; no training needed.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q: Why does speculative decoding NOT help at high batch sizes?

Answer: At high batch sizes ($\geq$64), autoregressive generation is already *compute-efficient*: the weight-read cost is amortized across many sequences. The arithmetic intensity approaches the roofline ridge point.
Speculative decoding adds overhead:

1. Draft generation cost (even if small model, it's not free at high batch)
2. Verification forward pass processes k extra tokens per sequence (batch $\times$ k tokens)
3. Memory for draft model or Medusa heads
4. Rejected tokens waste compute

At batch=1 (latency-bound, memory-bound): speculation turns 1 token/step into 3–4 tokens/step — huge win.
At batch=128 (already compute-efficient): the extra tokens from speculation barely help throughput because the GPU is already near-saturated. The overhead may even *reduce* throughput.
Rule: Speculative decoding is for latency (small batch); batching is for throughput (large batch). Don't combine them.
Review: *Chapter 2 (Systems Foundations for LLMs).*

28.18 Agentic RL Questions

Q: Why does standard RLHF (single-turn PPO/DPO) fail for multi-step agents?

Answer: Standard RLHF optimizes for single-turn quality: given a prompt, produce one good response. Multi-step agents face fundamentally different challenges:

1. **Credit assignment**: In a 50-step trajectory, which step caused the failure? Single-turn reward assigns credit to the entire response uniformly; multi-step needs *per-step* credit.
2. **Sparse rewards**: Success/failure only at trajectory end. PPO's GAE assumes intermediate rewards; without them, advantage estimates are noisy.
3. **Action space**: Actions are structured tool calls (JSON), not just token sequences. The model must learn syntax + semantics + strategy simultaneously.
4. **Non-stationarity**: The environment changes with each action (tool outputs modify state). Each step has a different "prompt" unlike single-turn where input is fixed.
5. **Exploration**: Agent must discover novel tool-use strategies, not just rephrase text.

Solution: Trajectory-level GRPO (rank complete trajectories), process reward models (per-step feedback), or filtered SFT on successful trajectories.
Review: *Chapter 12 (LLM Agentic Training).*

Q: Explain how GRPO is adapted for agentic training. What are the key differences from single-turn GRPO?

Answer: Single-turn GRPO: generate N responses to a prompt, rank by reward, compute advantages.
Agentic GRPO differences:

1. **Unit of generation**: A full *trajectory* (10–100 steps) instead of a single response. Each trajectory is one "sample" in the group.
2. **Reward**: Terminal (task success/failure) or trajectory-level (sum of step rewards). NOT per-token.
3. **Masking**: Only compute policy loss on the agent's outputs (reasoning + tool calls). Mask tool *outputs* (environment responses) from gradient computation.
4. **Group size**: Typically smaller ($N = 4$–8) because trajectories are expensive (many forward passes per trajectory).
5. **KL penalty**: Applied per-step to prevent drift from SFT policy at each decision point.
6. **Length normalization**: Normalize by number of agent *actions* (not tokens) to avoid penalizing thorough reasoning.

Review: *Chapters 7 and 12 (GRPO; LLM Agentic Training).*

Q: Compare STaR and Reflexion and ReAct for agents.

Answer:
STaR (Self-Taught Reasoner): Generate reasoning chains → filter by correctness → fine-tune on correct ones. *Use when*: You have verifiable tasks (math, code) and want to bootstrap reasoning from a base model without RL.
Reflexion: After failure, generate verbal feedback ("What went wrong?") → retry with reflection in context. No weight updates. *Use when*: Inference-time improvement; limited compute for training; tasks where self-diagnosis is possible.
ReAct: Interleave Reasoning (think) + Acting (tool use) in a structured loop. *Use when*: Multi-step tool use tasks; you need transparency (reasoning traces are interpretable); the agent must decide between thinking and acting.
Key differences:

	STaR	**Reflexion**	**ReAct**
Updates weights?	Yes (SFT)	No (in-context)	No (prompting)
Multi-step?	No (single reasoning chain)	Yes (retry loops)	Yes (think-act cycles)
Tools?	No	Optional	Yes (required)
Best for	Reasoning improvement	Error recovery	Tool-augmented tasks

Review: *Chapters 12 and 18 (LLM Agentic Training; Agent Design Patterns).*

Q: Why is GRPO preferred over PPO for a research agent?

Answer: For a research agent with 20–100 step trajectories:
PPO requires a value model: $V(s_t)$ must predict expected total reward from the current state. For research (where state = 128K tokens of context including papers, code, and results), training an accurate value function is extremely difficult — the value of "having read 3 papers and written partial code" is hard to predict.
GRPO avoids value estimation entirely: It generates N complete trajectories per research question and uses within-group ranking as the advantage. No need to predict intermediate value — just compare outcomes.
Additional reasons:

- Research quality is binary-ish (good report vs. bad report) — ranking is natural.
- Trajectories are long and expensive; GRPO's $N = 4$ is manageable; PPO would need many rollouts for stable value estimates.
- Terminal reward is sparse; GAE with sparse rewards gives noisy per-step advantages anyway.

Review: *Chapters 7 and 12 (GRPO; LLM Agentic Training).*

Q: Design a reward function for a coding agent. What reward hacking risks exist?

Answer: **Reward design**:

$$R = 0.5 \cdot R_{\text{tests}} + 0.2 \cdot R_{\text{quality}} + 0.2 \cdot R_{\text{efficiency}} + 0.1 \cdot R_{\text{safety}}$$

- R_{tests}: Fraction of unit tests passing (0–1). Ground-truth verifiable.
- R_{quality}: LLM judge on code style, documentation, maintainability.
- $R_{\text{efficiency}}$: $\max(0, 1 - \text{steps}/30)$ — bonus for finishing quickly.
- R_{safety}: No dangerous operations (rm -rf, network access outside sandbox).

Reward hacking risks:

1. **Hardcoded outputs**: Agent learns to print expected test outputs directly without computing them. *Fix*: Randomize test inputs; test on held-out cases.
2. **Test deletion**: Agent modifies/deletes failing tests. *Fix*: Sandbox tests as read-only.
3. **Trivial solutions**: Agent writes minimal code that passes tests but doesn't generalize. *Fix*: Large, diverse test suites; property-based testing.
4. **Efficiency gaming**: Agent skips reasoning steps to maximize efficiency bonus. *Fix*: Minimum quality threshold before efficiency bonus applies.

Review: *Chapters 9, 12, and 19 (Reward Model Training; LLM Agentic Training; Agentic Environments).*

28.19 Listwise Rewards and Advanced RM Questions

Q: Explain the Plackett-Luce model. How does it generalize Bradley-Terry?

Answer: Bradley-Terry models *pairwise* preferences: $P(y_1 \succ y_2) = \sigma(r(y_1) - r(y_2))$.
Plackett-Luce models *full rankings* of K items as sequential selection:

$$P(\pi) = \prod_{i=1}^{K} \frac{e^{r(y_{\pi(i)})}}{\sum_{j=i}^{K} e^{r(y_{\pi(j)})}}$$

Interpretation: Sequentially pick the best remaining item. Position 1 = softmax over all K; position 2 = softmax over remaining $K - 1$; etc.
Generalization: For $K = 2$, PL reduces exactly to BT: $P(y_1 \succ y_2) = \frac{e^{r(y_1)}}{e^{r(y_1)} + e^{r(y_2)}} = \sigma(r(y_1) - r(y_2))$.
Advantage: A ranking of $K = 8$ items provides $\binom{8}{2} = 28$ implicit pairwise comparisons plus relative margin information — much richer training signal than a single pair.
Review: *Chapter 9 (Reward Model Training).*

Q: What is a Process Reward Model (PRM) and when is it better than an Outcome Reward Model (ORM)?

Answer: **ORM**: Scores the final output only. $r(x, y_{\text{final}})$ = one scalar for the complete response.
PRM: Scores each *step* of reasoning. $r(x, y_{\text{step } t})$ = scalar per intermediate step.
PRM is better when:

1. **Long reasoning chains**: Math problems with 10+ steps. ORM can't tell which step went wrong; PRM provides per-step credit assignment.
2. **Search/verification**: PRM enables tree search (beam search over reasoning steps, prune branches with low step-reward).
3. **Training signal density**: PRM gives T rewards per trajectory (one per step) vs. ORM's single reward → lower variance advantage estimates.

ORM is better when: Tasks are short (single-turn); step boundaries are unclear; annotation cost for per-step labels is prohibitive.
PRM annotation: Can be automated via "Math-Shepherd" approach: for each step, complete the solution from that point multiple times. If completions from step t succeed but completions from step $t+1$ fail, step $t+1$ is likely wrong.
Review: *Chapters 9 and 13 (Reward Model Training; RL for Large Reasoning Models).*

28.20 RL for Large Reasoning Models Questions

Q: Why does DeepSeek-R1 not use a Process Reward Model despite training on long reasoning chains?

Answer: DeepSeek-R1 uses only **outcome-based rewards** (accuracy + format) for several reasons:

1. **Verifiable tasks**: Math and code have deterministic ground-truth answers. The binary accuracy reward provides sufficient signal even for long chains.
2. **PRM failure modes**: Step-level reward models introduce their own reward hacking — the model can learn to produce steps that "look correct" to the PRM without actually being correct.
3. **GRPO's group normalization**: By sampling G completions per prompt and normalizing advantages within each group, GRPO naturally provides relative signal about which reasoning *strategies* work, even without per-step rewards.
4. **Emergent self-correction**: With outcome-only rewards, the model learns to self-correct within chains (the "aha moment"), which wouldn't emerge if per-step rewards micromanage the reasoning process.

Key insight: The verifiability of the task domain is what makes PRMs unnecessary — for subjective tasks (creative writing), outcome-only rewards may not suffice.
Review: *Chapter 13 (RL for Large Reasoning Models).*

Q: Explain the test-time compute scaling law and its implications for model deployment

Answer: The test-time compute scaling law states:

$$\text{Accuracy}(C_{\text{train}}, C_{\text{test}}) \approx f(\alpha \log C_{\text{train}} + \beta \log C_{\text{test}})$$

Implications:

1. **Compute equivalence**: A 7B model with 64× more inference tokens can match a 70B

model with 1× tokens on reasoning tasks.

2. **Adaptive allocation**: Easy questions get short chains (cheap); hard questions get long chains (expensive). Average cost is lower than always using a large model.
3. **Deployment flexibility**: Instead of one large model, deploy a smaller reasoning model and scale inference compute per-query based on difficulty.
4. **Diminishing returns**: The log relationship means doubling test-time compute gives diminishing accuracy gains — there's an optimal allocation between training and inference compute.

The "overthinking" failure: Very long chains can *decrease* accuracy due to error accumulation and attention dilution. Optimal chain length depends on problem difficulty.
Review: *Chapter 13 (RL for Large Reasoning Models).*

Q: How does MCTS (Monte Carlo Tree Search) apply to LLM reasoning?

Answer: MCTS for reasoning treats each partial solution as a tree node:
Four phases per iteration:

1. **Selection**: Navigate from root using UCB: $\text{UCB}(s) = Q(s) + c\sqrt{\frac{\ln N(\text{parent})}{N(s)}}$
2. **Expansion**: Generate new reasoning steps (child nodes) from the LLM
3. **Simulation**: Complete the solution from the new node (rollout)
4. **Backpropagation**: Update Q-values along the path based on final correctness

Key differences from game MCTS:

- **Branching factor**: Reasoning has enormous branching factor (any next sentence is possible). Practical implementations use the LLM's top-k outputs to limit branches.
- **Value function**: A trained PRM estimates partial solution quality, replacing random rollouts.
- **Step granularity**: Each "step" might be one sentence, one equation, or one logical inference — choosing granularity matters.

Used by: AlphaProof (math olympiad), and hypothesized for OpenAI o1/o3's hidden reasoning.
Review: *Chapter 13 (RL for Large Reasoning Models).*

Q: Compare distillation vs direct RL for creating small reasoning models

Answer:
Distillation (DeepSeek-R1-Distill approach):

- Generate reasoning chains from large model (R1-671B)
- SFT small model on these chains
- Result: Small model mimics large model's reasoning *format*
- Pro: Cheap (just SFT). Con: May learn surface patterns not true reasoning.

Direct RL on small model:

- Train small model with GRPO/PPO against verifiable rewards

- Model discovers its own reasoning strategies
- Pro: Genuine capability. Con: Much more compute; may not converge for very small models.

Empirical finding: R1-Distill-7B (distilled) outperforms direct-RL-7B on most benchmarks. The reasoning chains from the large model provide such strong supervision that SFT alone is competitive. However, distilled models show less generalization to truly novel problem types.
Best practice: Distill first (cheap baseline), then optionally run RL on the distilled model for further gains ("distill + RL" combo used by Qwen).
Review: *Chapter 13 (RL for Large Reasoning Models).*

28.21 LLM Evaluation Questions

Q: Derive the ELO rating update rule and explain why Chatbot Arena uses it

Answer: **ELO derivation**:
Expected score of player A vs B: $E_A = \frac{1}{1+10^{(R_B-R_A)/400}}$ (logistic model).
After a match with actual score $S_A \in \{0, 0.5, 1\}$: $R'_A = R_A + K(S_A - E_A)$
The K-factor controls update magnitude (higher K = more reactive to recent results).
Why Chatbot Arena uses ELO:

1. **Transitivity**: If model A beats B and B beats C, ELO predicts A beats C. This gives a total ordering from pairwise comparisons.
2. **Online updates**: New models can be added without re-evaluating all pairs. Each new comparison updates ratings incrementally.
3. **Confidence**: After N comparisons, rating uncertainty shrinks as $O(1/\sqrt{N})$. Standard error: SE $\approx \frac{400}{\sqrt{N}}$.
4. **Human preference capture**: Real users provide honest preferences without needing to articulate criteria. The aggregate reveals true model quality.

Chatbot Arena specifics: Uses Bradley-Terry MLE (equivalent to ELO at convergence) with bootstrap confidence intervals. Style-controlled ELO removes length/formatting bias.
Review: *Chapter 14 (LLM Evaluation).*

Q: What is the pass@k metric for code generation and why is the unbiased estimator important?

Answer: **pass@k** = probability that at least one of k generated samples passes all test cases.
Naive (biased) estimator: Generate k samples, check if any passes. Problem: high variance, expensive (need many trials per problem).
Unbiased estimator (Chen et al., 2021): Generate $n \geq k$ samples, count c that pass:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

Why unbiased matters:

1. Generates n samples once (e.g., $n = 200$) and computes pass@1, pass@10, pass@100 from the same samples
2. No need to repeat the entire evaluation k times
3. Statistically exact (combinatorial argument: fraction of k-subsets with no correct sample)
4. Numerically stable computation via log-space: $\text{pass@}k = 1 - \exp\left(\sum_{i=0}^{k-1} \log(n-c-i) - \log(n-i)\right)$

Intuition: If 50/200 samples pass ($c = 50$, $n = 200$), pass@1 ≈ 0.25, pass@10 ≈ 0.94. The estimator counts what fraction of k-sized draws would contain at least one success.
Review: *Chapters 14 and 19 (LLM Evaluation; Agentic Environments).*

Q: How do you detect and mitigate benchmark contamination?

Answer: **Contamination**: Training data contains benchmark test examples (or close paraphrases), inflating scores.
Detection methods:

1. **N-gram overlap**: Check if training data contains exact or near-exact matches to test items. 8-gram overlap with >80% coverage is suspicious.
2. **Canary strings**: Insert unique identifiers in test sets; check if model can reproduce them.
3. **Rephrased benchmarks**: Create semantically equivalent but textually different versions of benchmarks. Large accuracy drops suggest memorization.
4. **Temporal analysis**: Model performance on pre-training-cutoff vs post-cutoff test items. Unusually high performance on old items suggests contamination.
5. **Membership inference**: Statistical tests for whether specific examples were in training data.

Mitigation:

- **Dynamic benchmarks**: Regularly generate new test items (LiveCodeBench, Chatbot Arena)
- **Private test sets**: Keep test items secret (LMSYS)
- **Decontamination during training**: Remove detected overlaps from training data
- **Report contamination analysis**: Disclose overlap metrics alongside benchmark scores

Review: *Chapter 14 (LLM Evaluation).*

Q: Explain position bias in LLM-as-Judge and how to mitigate it

Answer: **Position bias**: When using an LLM to judge two responses (A vs B), the model systematically prefers the response in a particular position (usually first or last), regardless of quality.
Empirical magnitude: GPT-4 shows 10–15% position bias; Claude shows 5–10%. Smaller models show larger bias.
Mitigation strategies:

1. **Position swapping**: Judge each pair twice (A-B and B-A). Final decision = majority. If disagreement, mark as "tie." This eliminates systematic position bias but doubles cost.
2. **Multi-judge panels**: Use 3+ different models as judges. Majority vote reduces individual model biases.
3. **Reference-guided**: Provide a rubric or reference answer. Judges score each response independently against the rubric, then compare scores (eliminates pairwise comparison entirely).
4. **Calibrated prompting**: Add explicit instruction: "The order of presentation is random and should not influence your judgment."

Additional biases: Verbosity bias (prefers longer responses), self-enhancement bias (models prefer their own outputs), authority bias (defers to responses that cite sources).

Review: *Chapter 14 (LLM Evaluation).*

28.22 Agentic Memory Questions

Q: Compare the four types of agentic memory and when each is critical

Answer:

Type	What it stores	Access pattern	Critical when
Working	Current context/scratchpad	Always in context	Complex multi-step reasoning
Episodic	Past experiences	Retrieved by similarity	Learning from past mistakes
Semantic	Facts and knowledge	Retrieved by concept	Domain-specific tasks
Procedural	Skills and patterns	Triggered by task type	Repeated tool-use

Key insight: These aren't independent — they interact. Episodic memory feeds semantic memory (generalizing from episodes to facts). Procedural memory is refined by episodic feedback (learning which tool sequences work). Working memory orchestrates retrieval from all other types.
MemGPT analogy: Working = hot (in-context), Episodic/Semantic = warm (vector store), Procedural = cold (archived policies). The agent itself decides when to page information in/out.
Review: *Chapter 16 (Agentic Memory Systems).*

Q: How does temporal decay work in memory retrieval and why is it important?

Answer: **Temporal decay** down-weights older memories during retrieval:

$$\text{score}(m) = \alpha \cdot \text{similarity}(q, m) + (1 - \alpha) \cdot \text{recency}(m)$$

where $\text{recency}(m) = e^{-\lambda \cdot \Delta t}$ with Δt = time since last access.
Why it's important:

1. **Relevance decay**: User preferences change. A preference from 6 months ago may be outdated.
2. **Contradiction resolution**: When old and new information conflict, recency bias naturally prefers current truth.
3. **Retrieval efficiency**: Without decay, memory grows unbounded and retrieval returns increasingly irrelevant ancient items.
4. **Cognitive plausibility**: Humans forget too — recent events are more accessible. This mirrors the spacing effect.

Access-based refresh: When a memory is retrieved and used, its timestamp updates (similar to LRU caching). Frequently-accessed memories stay "fresh" regardless of creation date.
Decay rate tuning: λ depends on domain. Customer service: high decay (preferences change fast). Legal/medical: low decay (facts persist). Can be learned via RL.
Review: *Chapter 16 (Agentic Memory Systems).*

Q: How can RL be used to train memory operations?

Answer: Memory operations (write/read/update/delete) can be actions in the agent's MDP:
Formulation:

- **State**: Current context + memory state
- **Actions**: Standard actions + `memory_write(key/value)`, `memory_read(query)`, `memory_delete(key)`

- **Reward**: Task success (did memory help?) + memory efficiency penalty (fewer reads = better)

What RL learns:

1. **What to store**: Important information (API keys/user preferences) vs ephemeral details
2. **When to retrieve**: Before answering domain questions vs during general chat
3. **Compression policy**: When to summarize old memories vs keep verbatim
4. **Forgetting**: When old information is stale and should be removed

Training signal: Counterfactual — "would the agent have succeeded if it hadn't stored/retrieved this memory?" Implemented via trajectory comparison: trajectories with good memory use get higher rewards.
Challenge: Delayed reward — storing information now may only help 100 steps later. Requires long-horizon credit assignment (GAE with high λ).
Review: *Chapters 12 and 16 (LLM Agentic Training; Agentic Memory Systems).*

28.23 Agent Orchestration Questions

Q: Explain the context budget problem and how to solve it with dynamic allocation

Answer: **The problem**: An agent has context window L tokens but needs space for:

$$C = S + M + T + H + R \leq L$$

where S = system prompt, M = memory/retrieved context, T = tool descriptions, H = conversation history, R = reserved for response.
As conversations grow, H increases and pushes out other components.
Dynamic allocation strategy:

1. **Fixed minimums**: $S_{\min}$, $R_{\min}$ are non-negotiable
2. **Adaptive history**: Summarize old turns when $H > H_{\max}$. Keep last k turns verbatim; summarize the rest.
3. **On-demand tools**: Only include tool descriptions relevant to current query (not all 50 tools). Use a classifier or embedding similarity to select top-k tools.
4. **Lazy memory**: Retrieve memory only when needed (after analyzing the query) rather than pre-loading.

Overflow handling: When total exceeds L even after compression:

- Drop least-important tool descriptions
- Aggressively summarize history to 1-sentence-per-turn
- Reduce memory slots
- If still over: truncate with warning to user

Pre-flight check: Always count tokens BEFORE calling the LLM. Never discover overflow at inference time.
Review: *Chapter 17 (Agent Harness – Context Management and Orchestration).*

Q: Compare ReAct vs Plan-and-Execute orchestration patterns

Answer:
ReAct (Reason + Act):

- Loop: Thought → Action → Observation → Thought → ...
- Each step decides the next action based on all previous observations
- **Pro**: Adaptive — can change direction based on tool outputs
- **Con**: Myopic — no upfront planning; can get stuck in loops; each LLM call sees entire history (expensive)

Plan-and-Execute:

- Phase 1: Generate full plan (list of steps)
- Phase 2: Execute steps sequentially (simpler executor; possibly cheaper model)
- Phase 3: Re-plan if execution fails
- **Pro**: Efficient (planning once is cheaper than reasoning every step); parallelizable independent steps
- **Con**: Brittle plans — if early steps fail the plan may be invalid. Re-planning adds latency.

When to use which:

- ReAct: Exploratory tasks; unknown environments; tasks where each step's result determines the next
- Plan-and-Execute: Well-defined tasks; known tool set; parallelizable sub-tasks; cost-sensitive deployments
- **Hybrid**: Plan at high level then ReAct within each plan step (LangGraph's recommended pattern)

Review: *Chapters 17 and 18 (Agent Harness; Agent Design Patterns).*

Q: How do you detect and prevent infinite loops in agent execution?

Answer: Agents can enter infinite loops when they repeat the same action expecting different results.
Detection methods:

1. **Max iteration guard**: Hard limit (e.g., 25 steps). Simple but loses work on genuinely long tasks.
2. **Action hash window**: Hash the last k (action/observation) pairs. If current hash matches a hash from the last w steps then loop detected.
3. **Semantic similarity**: Embed recent actions. If cosine similarity between consecutive actions exceeds threshold (>0.95) then likely stuck.
4. **Progress monitoring**: Define task-specific progress metrics. If no progress in N steps then intervene.

Recovery strategies:

1. **Inject hint**: Add system message: "You seem to be repeating actions. Try a different approach."

2. **Force different action**: Mask the repeated action from the action space for the next step.
3. **Escalate**: Return to user with partial results and ask for guidance.
4. **Backtrack**: Reset to a checkpoint before the loop began and try alternative path.

Best practice: Combine max iterations (safety net) + hash-based detection (early intervention) + graceful escalation (preserve user trust).
Review: *Chapters 17 and 18 (Agent Harness; Agent Design Patterns).*

28.24 MCP Protocol Questions

Q: Explain MCP's N+M architecture and why it matters for the agent ecosystem

Answer: **The N×M problem**: Without MCP, N agent frameworks must each implement integrations with M tools = $N \times M$ total integrations. Adding one new tool requires N implementations.
MCP's N+M solution: Standardize the interface. Each agent implements one MCP client (N total). Each tool implements one MCP server (M total). Total integrations = $N + M$.
Concrete example: 5 agent frameworks (LangChain/AutoGen/CrewAI/Claude/custom) × 20 tools (GitHub/Slack/DB/filesystem/...) = 100 integrations without MCP. With MCP: 5 clients + 20 servers = 25 implementations.
Why it matters:

1. **Tool reuse**: Build a tool server once; use from any MCP-compatible agent
2. **Agent portability**: Switch from Claude to a custom agent without rewriting tool integrations
3. **Ecosystem growth**: Lower barrier to adding new tools incentivizes the community to build more
4. **Composability**: Connect multiple servers to one agent dynamically at runtime

Analogy: USB standardized peripheral connections. Before USB: every device had a proprietary connector. After USB: one port fits all. MCP does the same for agent-tool connections.
Review: *Chapter 20 (Model Context Protocol).*

Q: What are MCP's four core primitives and when do you use each?

Answer:

Primitive	Direction	Purpose	Example
Tools	Client → Server	Execute actions	`create_issue`; `query_db`
Resources	Client → Server	Read data	File contents; DB records
Prompts	Client → Server	Get templates	"Summarize this PR" template
Sampling	Server → Client	Request LLM gen	Server asks LLM to classify

Key distinctions:

- **Tools vs Resources**: Tools have *side effects* (create/modify/delete). Resources are *read-only*. This distinction matters for safety — an agent can freely read resources but must get approval for tools.
- **Sampling** reverses the direction: normally the client (agent) calls the server (tool). With Sampling the server asks the client's LLM for help. Use case: a code analysis server needs the LLM to interpret a code snippet.
- **Prompts** are metadata (reusable templates) not execution. They help the agent formulate better tool calls.

Review: *Chapter 20 (Model Context Protocol).*

28.25 Agent Communication (A2A) Questions

Q: How does Google's A2A protocol differ from MCP and when do you need both?

Answer: **Core distinction**:

- **MCP**: Agent ↔ Tool (structured function calls with defined schemas)
- **A2A**: Agent ↔ Agent (opaque task delegation — you don't know how the other agent works)

Key A2A concepts:

- **Agent Cards**: JSON describing what an agent can do (like a resume). Discovery mechanism.
- **Opaque execution**: Requester doesn't see internal reasoning of the delegate. Just sends task and gets result.
- **Task lifecycle**: submitted → working → completed/failed (with streaming updates via SSE)

When you need both:

1. An orchestrator agent uses **A2A** to delegate "research this topic" to a research agent
2. The research agent uses **MCP** to call web search and file read and database tools
3. Results flow back via A2A to the orchestrator

Architecture: A2A sits at the *inter-agent* layer; MCP sits at the *agent-tool* layer. A complete system uses both: A2A for coordination between agents and MCP for each agent's tool access.
Review: *Chapters 20 and 22 (MCP; Agent-to-Agent Communication).*

Q: What is the Contract Net Protocol and how does it apply to LLM agents?

Answer: The **Contract Net Protocol** (CNP) is a task allocation mechanism from distributed AI:
Steps:

1. **Announce**: Manager broadcasts task description to all available agents
2. **Bid**: Agents assess their capability and submit bids (confidence; estimated cost; estimated time)
3. **Award**: Manager selects best bid(s) based on criteria (capability/cost/availability)
4. **Execute**: Winning agent(s) perform the task
5. **Report**: Agent reports results back to manager

For LLM agents:

- **Bidding = self-assessment**: Each agent LLM evaluates "can I do this task well?" and provides a confidence score. This requires calibrated self-knowledge.
- **Specialization emerges**: Code agents bid high on code tasks; research agents bid high on research tasks. No central routing logic needed.
- **Load balancing**: If one agent is busy (high estimated time) others win the contract.

- **Failure handling**: If awarded agent fails then re-announce to remaining agents (automatic failover).

Limitation for LLMs: LLMs often overestimate their capabilities (hallucinate confidence). Bids should incorporate track record (historical success rate on similar tasks) not just self-reported confidence.
Review: *Chapters 22 and 23 (A2A; Multi-Agent Systems).*

28.26 Multi-Agent Systems Questions

Q: Compare centralized vs decentralized multi-agent architectures for LLMs

Answer:
Centralized (Supervisor):

- One orchestrator LLM routes tasks to specialist workers
- Clear control flow; easy to debug (inspect supervisor decisions)
- Single point of failure; supervisor becomes token bottleneck
- Best for: well-defined workflows; small agent teams (3–5 agents)

Decentralized (Peer-to-Peer):

- Agents communicate directly; no central coordinator
- Resilient (no single point of failure); scales horizontally
- Hard to debug (emergent behavior); potential for conflicts and deadlocks
- Communication scales $O(n^2)$ without structure
- Best for: resilient systems; large agent populations; creative tasks where emergent behavior is desired

Hybrid (Hierarchical): Tree structure with sub-managers. Combines benefits: local autonomy within groups and global coordination at the top. Communication scales $O(n \log n)$.
Decision framework: Use centralized if you need predictability and auditability. Use decentralized if you need resilience and creativity. Use hierarchical for large (>10 agent) systems.
Review: *Chapter 23 (Multi-Agent Systems).*

Q: What is CTDE and why is it important for training multi-agent LLM systems?

Answer: **CTDE** = Centralized Training; Decentralized Execution.
The problem: In multi-agent RL each agent's environment is non-stationary (other agents are changing their policies simultaneously). This makes independent training unstable.
CTDE solution:

- **During training**: A centralized critic has access to all agents' observations and actions: $V(s_1, s_2, \ldots, s_n, a_1, a_2, \ldots, a_n)$. This stabilizes training by removing non-stationarity from the value function.
- **During execution**: Each agent acts based only on its own observation: $a_i = \pi_i(o_i)$. No communication overhead at inference time.

For LLM agents: The centralized critic can be a reward model that evaluates the *joint* output of all agents (e.g., did the team of agents produce a correct software system?) while each agent is trained to maximize its contribution to the team reward using counterfactual credit assignment.

Practical challenge: Full CTDE requires all agents to train simultaneously with shared state — expensive for LLMs. Approximations: train agents in rounds (freeze others and train one) or use population-based training with periodic synchronization.
Review: *Chapter 23 (Multi-Agent Systems).*

28.27 Agent Development Framework Questions

Q: Compare LangGraph vs AutoGen vs CrewAI for building multi-agent systems

Answer:

Dimension	LangGraph	AutoGen / CrewAI
Orchestration	Explicit state graph (nodes + edges)	Implicit (conversation / role-based)
State mgmt	TypedDict schemas; checkpointing	Conversation history as state
Multi-agent	Graph with conditional routing	GroupChat / Crew
Debugging	Graph visualization; step replay	Chat logs
HITL	First-class (interrupt nodes)	Via approval tools
Production	LangGraph Cloud; persistence	Limited (AutoGen); growing (CrewAI)
Learning curve	High (graph concepts)	Low (AutoGen); Very low (CrewAI)

Choose LangGraph when: You need fine-grained control; complex conditional flows; production deployment with persistence and human-in-the-loop.
Choose AutoGen when: Rapid prototyping of multi-agent conversations; code execution agents; research experimentation.
Choose CrewAI when: Simple role-based teams; sequential task execution; quick demos; minimal code.
Choose none (custom) when: You need maximum performance/control; don't want framework lock-in; or have non-standard orchestration patterns.
Review: *Chapter 24 (Agent Development Frameworks).*

Q: How do you test and evaluate an agent system in production?

Answer: Agent testing follows a **testing pyramid**:
Level 1 — Unit Tests (fast; many):

- Test individual tools in isolation (mock LLM; verify tool logic)
- Test prompt templates (given context; verify correct prompt construction)
- Test parsers (given LLM output; verify correct extraction)

Level 2 — Integration Tests (medium speed):

- Test complete agent loops with deterministic inputs
- "Golden trajectory" tests: known-good execution traces that must reproduce
- Tool chain tests: verify multi-tool sequences work end-to-end

Level 3 — Behavioral Tests (slow; few):

- Does the agent follow safety constraints? (adversarial inputs)
- Does it ask for clarification when appropriate?

- Does it stay within token/cost budgets?

Production evaluation:

- **A/B testing**: Route 5% of traffic to new agent version
- **Shadow mode**: Run new agent alongside old; compare outputs without serving
- **LLM-as-judge**: Automated quality scoring of agent responses
- **User satisfaction**: Thumbs up/down; task completion rate; time-to-resolution

Key metric: **Task Success Rate** (TSR) — fraction of tasks the agent completes correctly without human intervention.
Review: *Chapters 14 and 24 (LLM Evaluation; Agent Development Frameworks).*

28.28 Agentic Environments Questions

Q: Design a reward function for a web browsing agent environment

Answer: For WebArena-style tasks (e.g., "find the cheapest flight from NYC to SF on Dec 15"):
Sparse reward (simple but hard to learn from):

$$r = \begin{cases} 1 & \text{if final page/state matches ground truth} \\ 0 & \text{otherwise} \end{cases}$$

Dense reward (better for training; harder to design):

1. **Progress reward**: $+0.1$ for each page that brings agent closer to goal (measured by text similarity to target state)
2. **Efficiency penalty**: -0.01 per action (encourages shorter trajectories)
3. **Milestone rewards**: $+0.3$ for reaching intermediate goals (e.g., navigating to flight search page)
4. **Invalid action penalty**: -0.05 for actions that produce errors (404; form validation failures)

Potential-based shaping (preserves optimal policy):

$$r_{\text{shaped}}(s, a, s') = r(s, a, s') + \gamma\Phi(s') - \Phi(s)$$

where $\Phi(s) = -\text{min\_steps\_to\_goal}(s)$ (estimated by heuristic or learned value function).
Challenges: Partial observability (can't always tell if you're closer to goal); stochastic environments (page content changes); reward hacking (agent finds shortcuts that satisfy reward but not user intent).
Review: *Chapters 12 and 19 (LLM Agentic Training; Agentic Environments).*

Q: What makes SWE-bench a particularly challenging agent benchmark?

Answer: SWE-bench tests agents on real GitHub issues from popular Python repositories:
Why it's hard:

1. **Repository-scale context**: Agent must understand codebases with 100K+ lines. Cannot fit in context window — must explore and search and navigate.
2. **Underspecified tasks**: Issues are written by humans with implicit context. Agent must

infer what's actually needed.

3. **Multi-file edits**: Solutions often span multiple files with cascading dependencies.
4. **Test verification**: Must pass existing tests AND new tests that verify the fix.
5. **No hand-holding**: Unlike HumanEval (single function) SWE-bench requires full software engineering workflow: read issue → explore code → localize bug → implement fix → verify.

State of the art (2024–2025): Best agents solve ∼50% of SWE-bench Verified (curated subset). Full SWE-bench: ∼30%.
Key insight for training: SWE-bench exposes the gap between "coding ability" (writing correct functions) and "software engineering ability" (understanding systems; navigating codebases; making minimal changes). RL training on SWE-bench-style environments teaches agents exploration and planning strategies not just code generation.
Review: *Chapter 19 (Agentic Environments and Benchmarks).*

28.29 Agentic UI Framework Questions

Q: Compare chat-based vs canvas-based UI paradigms for agents

Answer:
Chat-based (ChatGPT; Claude default):

- Linear message stream: user → assistant → user → ...
- **Pro**: Familiar UX; natural for exploration and Q&A; easy to implement
- **Con**: Generated artifacts (code/documents) are buried in conversation. Hard to iterate on a specific artifact. Context gets lost in long conversations.

Canvas/Artifact-based (Claude Artifacts; ChatGPT Canvas; Cursor):

- Side panel displays generated content; chat panel for instructions
- Agent can create and edit and iterate on persistent artifacts
- **Pro**: Artifacts persist independently of chat. Direct editing by user. Version history.
- **Con**: More complex UI; requires artifact type detection; harder to implement streaming to both panels.

When to use which:

- Chat: brainstorming; Q&A; quick tasks; mobile interfaces
- Canvas: code generation; document writing; data analysis — any task with persistent output that needs iteration
- **Hybrid** (most modern UIs): Chat by default; auto-elevate to canvas when detecting code/document/visualization output

For agent training: The UI paradigm affects the reward signal. Canvas UIs provide explicit edit feedback (user modifies the artifact) which can be used for online learning.
Review: *Chapter 25 (Agentic UI Frameworks).*

Q: How do you design approval gates for human-in-the-loop agent systems?

Answer: Approval gates pause agent execution at critical points for human review.
Three-tier model:

1. **Auto-approve** (no gate): Safe reversible actions. Read operations; searches; calculations.
2. **Notify** (soft gate): Potentially impactful but recoverable. Send email; create draft; modify file. Agent proceeds but user is notified and can undo.
3. **Block** (hard gate): Irreversible or high-stakes. Delete data; send money; publish content; execute code with side effects. Agent MUST wait for explicit approval.

Design principles:

- **Minimize interruptions**: Too many gates = user abandons the agent. The 3-tier model lets most actions flow while catching dangerous ones.
- **Show context**: At approval gate display: what action; why (agent's reasoning); what will change; how to undo.
- **Batch approvals**: If agent needs 5 file writes present them together not one by one.
- **Timeout handling**: If user doesn't respond within T minutes either retry notification or proceed with safe default or abort gracefully.
- **Learning from approvals**: Track approval/rejection patterns. If users always approve a certain action type consider auto-promoting it.

Implementation: Tool annotations (MCP's `destructiveHint` and `readOnlyHint`) drive automatic gate assignment. Custom rules can override based on context.
Review: *Chapters 17 and 25 (Agent Harness; Agentic UI Frameworks).*

28.30 RAG and Agentic RAG Questions